
Product Design Practise Project Report

Subject Code : ID - 351



Project Report : FPGA Based Secure Transfer of Images with
Cloud Using BFV Homomorphic Encryption

submitted by :

Divyansh Tiwari

Roll Number : 123EC0039

Sohan Maity

Roll Number : 523EC0001

Vikramaditya Singh

Roll Number : 123EC0042

Pranjal Upadhyay

Roll Number : 523EC0012

Electronics and Communications Engineering

Indian Institute of Information Technology Design and Manufacturing, Kurnool

Submission Date : March 30th , 2026

FPGA Based Secure Data Transfer of Images with Cloud Using BFV Homomorphic Encryption

Acknowledgements

We would like to express our deepest gratitude to our Project Supervisor, **Dr. Mohamed Asan Basiri M** (Asst Prof Grade 1, Dept of ECE), for his invaluable guidance, continuous support, and constructive feedback throughout the course of this project.

We are highly indebted to our Course Instructor, **Dr. Elangovan K** (Asst Prof), and **Dr. Vinay Kumar Tiwari** (Asst Prof), for their valuable insights and guidance during the evaluation process.

Our sincere thanks go to the **Head of the Department (HOD) of ECE** for providing us with seamless access to the laboratories. We also extend our gratitude to the **Lab Technicians, PhD scholars, and JRFs** who patiently helped us understand the inner workings of the homomorphic algorithms.

Furthermore, we wish to thank all our **friends** for their constant motivation, encouragement, and moral support, which were crucial in bringing this project to completion.

Finally, we express our profound gratitude to **Indian Institute of Information Technology Design and Manufacturing (IIITDM), Kurnool** for providing all the necessary facilities, computing equipment, simulation tools, and robust platforms required to turn this project into a success.

Vikramaditya Singh
Roll Number: **123EC0042**

Sohan Maity
Roll Number: **523EC0001**

Divyansh Tiwari
Roll Number: **123EC0039**

Pranjal Upadhyay
Roll Number: **523EC0012**

FPGA Based Secure Data Transfer of Images with Cloud Using BFV Homomorphic Encryption

Contents

Acknowledgements	1
Abstract	1
1 Project Objective	2
1.1 Enhancements to the Base Project	2
1.2 Source Code and Demonstrations	2
2 Introduction and Motivation	3
2.1 The Trust Barrier in Cloud Computing	3
2.2 Fully Homomorphic Encryption (FHE)	3
3 Cryptographic Theory and Algorithmic Flow	5
3.1 Mathematical Foundations and Parameters	5
3.2 Key Generation	5
3.3 Linear Feedback Shift Register (LFSR) for Noise Generation	5
3.4 BFV Encryption Core Algorithm	6
3.5 BFV Decryption Algorithm	6
3.6 Cryptographic Theory and Algorithm Overview	6
4 FPGA Hardware Architecture and RTL Design	7
4.1 Top Level System Architecture	7
4.1.1 Module Hierarchy	8
4.2 Encryption	8
4.2.1 Image Preprocessing Pipeline (Python)	8
4.2.2 Finite State Machine (FSM) Implementation	9
4.2.3 Modular Arithmetic Units	9
4.2.4 Vivado Encryption IP Block Design	9
4.2.5 Behavioral Simulation and Verification	10
4.2.6 FPGA Implementation Results	11
4.2.7 Vitis Application Development	13
4.2.8 FPGA Performance Specifications Summary	13
4.3 Cloud Transfer	14
4.3.1 UART Capture Script	14
4.3.2 Secure Cloud Gateway Architecture	14
4.4 Decryption	14
4.4.1 Decryption Hardware Architecture	15
4.4.2 Functional Decryption Verification	15
4.4.3 Condition 2: Decryption with an Incorrect Cipher Key	16
5 Phase 2: ASIC SoC Design – RTL to GDSII Flow	18
5.1 ASIC Design Flow Overview	18
5.2 SoC Design Parameters	18
5.3 Pipelined FSM Architecture for ASIC	18
5.4 SDC Timing Constraints	18
5.5 Pre-Synthesis RTL Simulation	19

5.6	180nm TSMC Synthesis Results (Cadence Genus)	19
5.7	90nm TSMC Synthesis Results (Cadence Genus)	21
5.8	Technology Node Comparison	23
5.9	Physical Design – Cadence Innovus (180nm)	23
5.9.1	Floorplan	23
5.9.2	Power Network	24
5.9.3	Special Route and NanoRoute	24
5.9.4	Final Chip Layout	24
5.10	Signoff Verification Reports	25
6	BFV Encryption Gateway – Web Application	26
6.1	Purpose and Scope	26
6.2	Technology Stack	26
6.3	Architecture and Features	26
6.4	Platform Security Features	26
6.5	Infrastructure Pipeline	27
6.6	Client Dashboards and Interfaces	27
7	Consolidated Experimental Results	30
7.1	FPGA Implementation Summary	30
7.2	ASIC Synthesis Comparison	31
8	Applications	31
9	Conclusion and Future Work	32
9.1	Conclusion	32
9.2	Future Scope	33
10	Appendix A: FPGA Hardware RTL Codes	34
10.1	Modular Adder (bfv_mod_adder.v)	34
10.2	Modular Multiplier (bfv_mod_mult.v)	34
10.3	Key Generation Core (bfv_keygen.v)	34
10.4	Encryption Core (bfv_encrypt_core.v)	35
10.5	Top-Level FSM Controller (bfv_top.v – FPGA Version)	36
11	Appendix B: Software and Interfacing Scripts	38
11.1	Image Preprocessing Script (Image_converter.py)	38
11.2	Vitis C Memory Header Converter (convert_image.py)	38
11.3	UART Capture Script (capture_uart.py)	38
11.4	GitHub Cloud Sync Script (push_to_gist.py)	39
11.5	Decryption and Verification Script (decrypt.py)	39
12	Appendix C: ASIC Synthesis Scripts	41
12.1	Genus Synthesis Script (run_genus_pipelined_180.tcl)	41
12.2	SDC Timing Constraints (bfv_pipelined_180nm.sdc)	41
12.3	Testbench (tb_bfv_top.v)	42
	References	43

FPGA Based Secure Data Transfer of Images with Cloud Using BFV Homomorphic Encryption

Abstract

Cloud computing offers immense storage and processing power for image data, but traditional encryption exposes data privacy breaches during cloud processing. Standard encryption schemes, such as AES or RSA, require data to be decrypted before the cloud can compute it, creating a fundamental “Trust Barrier.” This project presents a hardware-accelerated, privacy-preserving scheme utilizing the Brakerski/Fan-Vercauteren (BFV) Fully Homomorphic Encryption (FHE) scheme implemented on a Xilinx Zynq-7000 Field Programmable Gate Array (FPGA).

The proposed system allows image data to be securely transferred, stored, and evaluated on the cloud (utilizing platforms such as Supabase and GitHub Gists) without ever exposing the raw pixel data. By developing custom AXI4-Lite IP cores for both encryption and decryption, the computational bottleneck inherent in software-based FHE is eliminated. The system successfully processes a 64×64 grayscale image (4096 pixels), transmitting the securely packed 28-bit ciphertexts over UART. A comprehensive software-hardware co-design approach ensures exact pixel-perfect retrieval (0-255 integer range) with zero data loss, establishing a secure end-to-end IoT bridging pipeline.

Furthermore, the FPGA-validated RTL was migrated to a full ASIC flow using Cadence Genus for synthesis and Cadence Innovus for Place-and-Route on both TSMC 180nm and 90nm technology nodes, demonstrating silicon-readiness of the BFV accelerator as a standalone System-on-Chip (SoC). A dedicated web application (BFV Encryption Gateway) was also developed to provide a client-side interface for managing image transfers and encrypted file retrieval.

1 Project Objective

The primary deliverables and objectives of this product design practice center explicitly on establishing the cryptographic core baseline:

1. **RTL Core Optimization:** Designing an optimized BFV Encryption IP for the Zynq-7000 FPGA SoC to eliminate software latency.
2. **Hardware Prototype:** Establishing an FPGA setup capable of processing captured images and transmitting them securely via UART.
3. **Cloud Dashboard & Integration:** Creating a secure IoT bridge to cloud environments (Supabase and GitHub Gists) to autonomously visualize encrypted streams.
4. **Recovery Tool:** Developing client-side verification scripts to actively retrieve and perfectly reconstruct the original mathematical images.

1.1 Enhancements to the Base Project

Beyond the mandatory fundamental scope of the hardware prototype, extreme emphasis was placed on elevating the architecture into a production-grade systemic framework:

- **ASIC SoC Tape-Out Readiness:** Migrating the validated FPGA RTL directly to a pristine dedicated ASIC flow using Cadence Genus and Innovus for TSMC 180nm and 90nm process nodes, serving to solidify explicit physical silicon realization.
- **Web Application Architecture:** Developing a dedicated BFV Encryption Gateway. The robust web portal leverages Next.js and Supabase forming a resilient architectural container granting dynamic client-side image management and strict role-based encrypted file retrieval parameters.

1.2 Source Code and Demonstrations

To explore the physical implementations and structural flow of the overall project in exhaustive detail, the following comprehensive resources are maintained and externally accessible:

Interactive Project Resources & Source Access

Demonstration Videos:

- [Video of the basic flow of algorithms of the BFV Homomorphic Encryption project](#)
- [Video for the complete product with User Interface](#)

Open-Source Code Repositories:

- [GitHub Repository for SoC Design](#)
- [GitHub Repository for Web Interface](#)

Live Web Application Deployment:

- [BFV Encryption Gateway Live Portal \(Fully Functional Interface\)](#)

2 Introduction and Motivation

2.1 The Trust Barrier in Cloud Computing

In the modern digital infrastructure, sensitive visual data—ranging from medical imaging to satellite reconnaissance—is frequently offloaded to cloud environments for storage and machine learning inference. However, conventional cryptographic protocols exhibit a severe limitation: data must be decrypted in the cloud's memory before the CPU or GPU can perform operations on it. This creates a vulnerability window where malicious actors or compromised server administrators can extract raw, unencrypted pixels.

2.2 Fully Homomorphic Encryption (FHE)

Homomorphic Encryption is a revolutionary cryptographic paradigm that allows mathematical operations to be performed directly on ciphertexts. When the processed ciphertext is decrypted, the result is identical to the output of the operations performed on the plaintext.

The rationale for selecting the BFV scheme over other homomorphic algorithms (such as Paillier, RSA, or CKKS) includes the following factors:

1. **Arbitrary Operations:** Unlike Partially Homomorphic Encryption (PHE) schemes like RSA (which only supports multiplication) or Paillier (which only supports addition), BFV supports both homomorphic addition and multiplication, which are strict prerequisites for image processing filters and neural networks.
2. **Exact Integer Retrieval:** Unlike CKKS, which utilizes floating-point approximations resulting in precision loss, BFV guarantees exact integer arithmetic. This is critical for exact 8-bit grayscale pixel retrieval (0-255).
3. **Hardware Native:** BFV is based on the Ring Learning With Errors (Ring-LWE) problem, making it highly amenable to hardware acceleration via polynomial multiplication and modular arithmetic circuits.

To better understand the flow of data within this cryptosystem, the entire lifecycle of the BFV encryption scheme is illustrated in Figure 1. The architecture features a clear separation of concerns between the untrusted cloud environment and the secure local edge clients. The plaintext message (in our case, image pixels) undergoes encryption on the sender's client device. The resulting ciphertext is then securely transmitted to the cloud, where homomorphic evaluation (such as mathematical filtering or machine learning inference) takes place natively on the encrypted polynomials without ever exposing the raw data. Finally, the evaluated ciphertext is retrieved by the recipient and decrypted locally using the secret key to reconstruct the processed plaintext result.

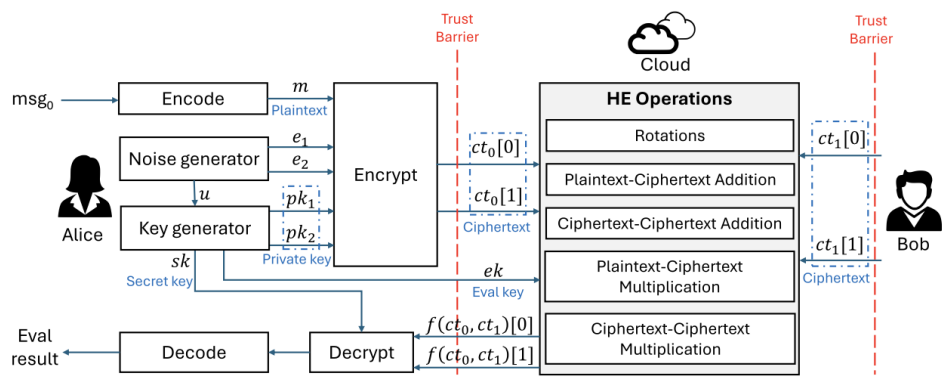


FIGURE 1. Complete diagram block of the BFV Homomorphic Encryption scheme.

Figure 1: Complete diagram block of the BFV Homomorphic Encryption scheme showcasing the data lifecycle from client-side encryption, to cloud-side homomorphic evaluation, and finally client-side secure decryption.

To explicitly contextualize the broader necessity and applicability of this specialized hardware acceleration, Figure 2 encapsulates a high-level systemic overview of the project’s ambition. This visual sets the foundational stage for exploring how dedicated Silicon IP provides unparalleled efficiency when handling exhaustive sequential polynomial evaluations compared to generic software-based CPU execution.



Figure 2: Introductory schematic representing the foundational motivation bridging complex Fully Homomorphic Encryption algorithms from conceptual mathematics to tangible edge silicon deployments.

3 Cryptographic Theory and Algorithmic Flow

3.1 Mathematical Foundations and Parameters

The hardware implementation strictly adheres to the mathematical foundations of the BFV scheme. To balance security, noise budget, and hardware resource utilization, the following constraints were configured:

- **Ciphertext Modulus (Q):** 12289 (14-bit word size, NTT-friendly prime).
- **Plaintext Modulus (T):** 256 (Perfectly encapsulates the 8-bit grayscale pixel range).
- **Polynomial Degree (N):** 4096 (Matches the 64×64 image size).
- **Scaling Factor (Δ):** $\lfloor Q/T \rfloor = \lfloor 12289/256 \rfloor = 48$.
- **Secret Key (SK):** Ternary value $\in \{0, +1, -1\}$, mapped to $\{0, 1, Q - 1\}$ in $\mathbb{Z}_q$.

3.2 Key Generation

The BFV public key pair (PK_1, PK_2) is derived from:

$$PK_1 = [-(a \cdot SK + e)] \pmod{Q} \quad (1)$$

$$PK_2 = a \quad (2)$$

where a is a random polynomial coefficient in $\mathbb{Z}_q$ and e is a small error polynomial sampled from the chi distribution, bounded by the noise budget.

3.3 Linear Feedback Shift Register (LFSR) for Noise Generation

To securely encrypt the data, ephemeral keys (u) and noise components (e_1, e_2) are required for every pixel. To minimize area, a 16-bit LFSR pseudo-random number generator is implemented in hardware utilizing the maximal-length polynomial $P(x) = x^{16} + x^{14} + x^{13} + x^{11} + 1$. The error terms are strictly clipped to satisfy the BFV noise budget constraint:

$$\|v\| = e_{key} + e_1 + e_2 \leq 3 + 10 + 10 = 23 < \frac{Q}{2T} = 24 \quad (3)$$

Algorithm 1 16-bit LFSR and BFV Parameter Generation

```

1: Initialize:  $lfsr \leftarrow 16'hACE1$ 
2: On posedge clock:
3:    $feedback \leftarrow lfsr[15] \oplus lfsr[13] \oplus lfsr[12] \oplus lfsr[10]$ 
4:    $lfsr \leftarrow \{lfsr[14 : 0], feedback\}$ 
5:
6: Extract Ephemeral Key ( $u$ ): ▷ Mapped to  $\mathbb{Z}_q\{0, 1, Q - 1\}$ 
7:    $u\_sel \leftarrow lfsr[1 : 0]$ 
8:   if  $u\_sel == 01$  then  $u \leftarrow 1$ 
9:   else if  $u\_sel == 10$  then  $u \leftarrow 12288$  ▷ Represents -1
10:  else  $u \leftarrow 0$ 
11:
12: Extract Error Terms ( $e_1, e_2$ ):
13:    $e1\_raw \leftarrow lfsr[4 : 0]; \quad e2\_raw \leftarrow lfsr[9 : 5]$ 
14:    $e1 \leftarrow \min(e1\_raw, 10)$  ▷ Clipped to 10 to prevent decryption failure
15:    $e2 \leftarrow \min(e2\_raw, 10)$ 

```

3.4 BFV Encryption Core Algorithm

For each plaintext pixel m , the hardware computes a ciphertext pair (C_1, C_2) by scaling the message, applying the noise, and multiplying by the public keys:

$$C_1 = [PK_1 \cdot u + e_1 + \Delta \cdot m] \pmod{Q} \quad (4)$$

$$C_2 = [PK_2 \cdot u + e_2] \pmod{Q} \quad (5)$$

3.5 BFV Decryption Algorithm

The decryption process reverses the encryption by computing the inner product and rounding:

$$inner = (C_1 + C_2 \cdot SK) \pmod{Q} \quad (6)$$

$$m = \left\lfloor \frac{inner \cdot T + \lfloor Q/2 \rfloor}{Q} \right\rfloor \pmod{T} \quad (7)$$

3.6 Cryptographic Theory and Algorithm Overview

In summary, the Brakerski-Fan-Vercauteren (BFV) scheme is a fully homomorphic encryption framework founded on the post-quantum Ring Learning With Errors (RLWE) hardness assumption. It operates over polynomial rings, mapping finite plaintext messages into noisy ciphertext spaces. This unique structure allows arithmetic computations—such as addition and multiplication—to be securely evaluated directly on the encrypted data without ever requiring mathematical access to the secret key. The deliberately injected noise guarantees robust semantic security and is dynamically managed throughout all homomorphic operations. Ultimately, during decryption, the receiver's private secret key is utilized to systematically strip away this accumulated noise envelope, securely restoring the exact original plaintext message via precise modulus switching.

4 FPGA Hardware Architecture and RTL Design

4.1 Top Level System Architecture

The system leverages a Hardware/Software co-design approach on the Xilinx Zynq-7000 (xc7z020clg484-1) SoC. The ARM Cortex-A9 Processing System (PS) manages the control flow, orchestrates UART transmission, and feeds image data. The Programmable Logic (PL) contains the custom-built `bfv_axi_encrypt` IP core.

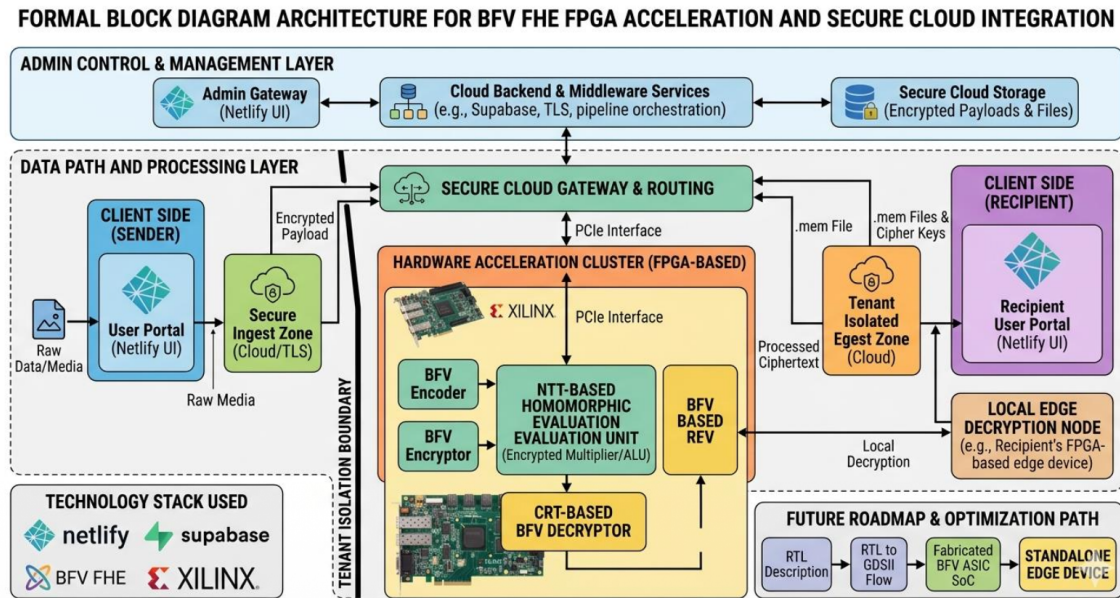


Figure 3: Final Architecture Block diagram of the system detailing the hardware-software split between the ARM Cortex-A9 Processing System and the FPGA Programmable Logic components.

As depicted in Figure 3, the processing pipeline starts from the Host PC where the Python script converts the real image into memory files. The data is transferred to the ARM processor on the testbed which uses the AXI interconnect to relay control signals to the custom hardware IP. The actual mathematical operations are accelerated natively in the PL utilizing hardened DSP slices for modular arithmetic.

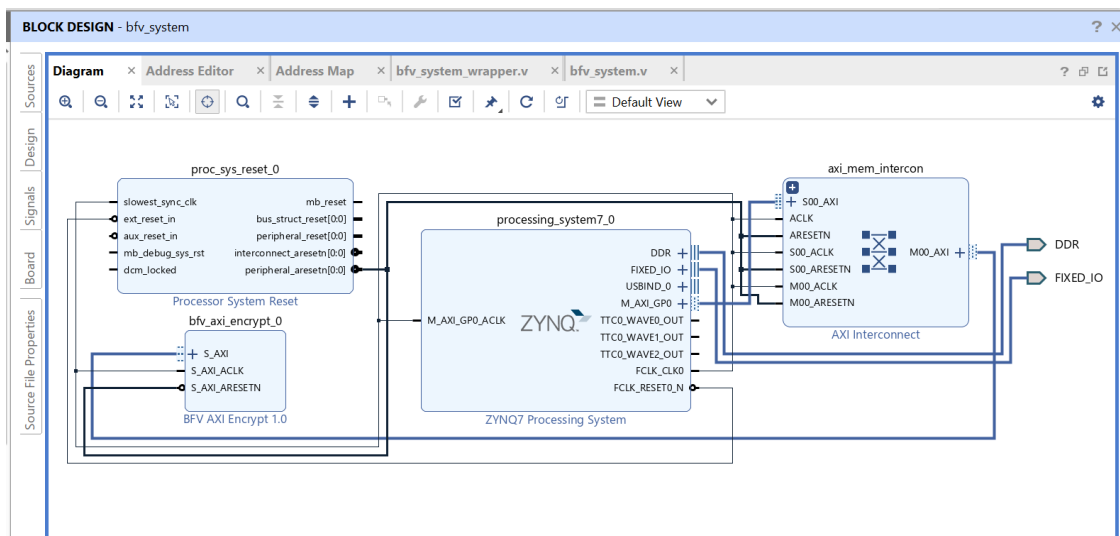


Figure 4: Top-Level Encryption Block Diagram showing Image BRAM, BFV Encryption Core, and Control Logic interfacing with the AXI-Lite bus.

Zooming further into the PL logic, Figure 4 delineates the internal organization of the custom IP core. It

features a local dual-port Block RAM (BRAM) to store the image payload temporarily, an FSM-driven controller, and the specialized BFV arithmetic core responsible for generating the ciphertext pairs.

4.1.1 Module Hierarchy

The RTL design follows a strict modular hierarchy:

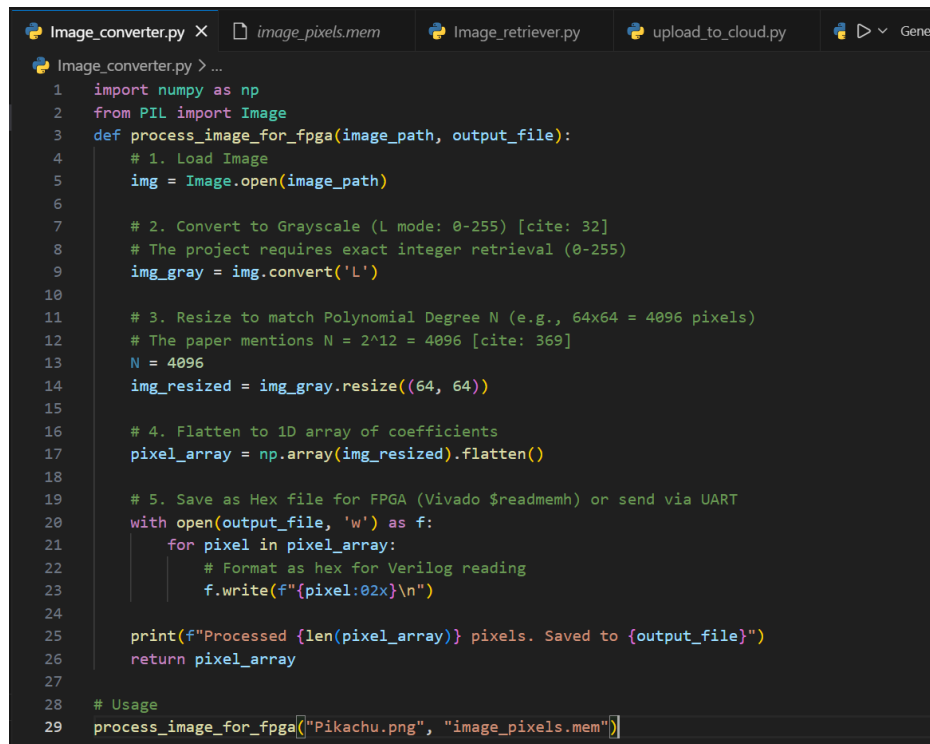
Module	Description
bfv_top	Top-level FSM controller with pixel memory and ciphertext storage
bfv_keygen	Public key generation: $PK_1 = -(a \cdot SK + e) \bmod q$, $PK_2 = a$
bfv_encrypt_core	BFV encryption: computes C_1 , C_2 from plaintext and public key
bfv_mod_mult	Modular multiplier with Barrett reduction (mod 12289)
bfv_mod_adder	Modular adder with single-cycle conditional subtraction

Table 1: RTL Module Hierarchy and Description.

4.2 Encryption

4.2.1 Image Preprocessing Pipeline (Python)

To feed visual data into the FPGA, raw images cannot be directly transmitted. A Python script leverages the PIL (Pillow) library to load the image, convert it to 'L' mode (8-bit grayscale), resize it to 64×64 , flatten it, and export it as a memory (.mem) file suitable for Vivado's \$readmemh directive.



```

Image_converter.py X image_pixels.mem Image_retriever.py upload_to_cloud.py
Image_converter.py > ...
1  import numpy as np
2  from PIL import Image
3  def process_image_for_fpga(image_path, output_file):
4      # 1. Load Image
5      img = Image.open(image_path)
6
7      # 2. Convert to Grayscale (L mode: 0-255) [cite: 32]
8      # The project requires exact integer retrieval (0-255)
9      img_gray = img.convert('L')
10
11     # 3. Resize to match Polynomial Degree N (e.g., 64x64 = 4096 pixels)
12     # The paper mentions N = 2^12 = 4096 [cite: 369]
13     N = 4096
14     img_resized = img_gray.resize((64, 64))
15
16     # 4. Flatten to 1D array of coefficients
17     pixel_array = np.array(img_resized).flatten()
18
19     # 5. Save as Hex file for FPGA (Vivado $readmemh) or send via UART
20     with open(output_file, 'w') as f:
21         for pixel in pixel_array:
22             # Format as hex for Verilog reading
23             f.write(f"{pixel:02x}\n")
24
25     print(f"Processed {len(pixel_array)} pixels. Saved to {output_file}")
26     return pixel_array
27
28 # Usage
29 process_image_for_fpga("Pikachu.png", "image_pixels.mem")

```

Figure 5: Python Image-to-MEM File Conversion terminal output capturing the exact flattening and file serialization operations ensuring compatibility with the target FPGA hardware boundaries.

The terminal prompt in Figure 5 reveals the instantaneous 4096-iteration parsing phase, serving as the critical bridge transforming standard user visual files into purely structural matrices accessible by the RTL constraint bounds.

4.2.2 Finite State Machine (FSM) Implementation

To maintain an optimal area-time tradeoff, the hardware utilizes a single shared combinatorial encryption core. A highly structured 7-state FSM iteratively drives the 4096 pixels through this core. The total clock cycles required are 16,386 clocks, taking approximately $\approx 164\mu s$ at 100 MHz.

FSM States: IDLE $\rightarrow$ KEYGEN $\rightarrow$ LOAD_PIX $\rightarrow$ COMPUTE $\rightarrow$ STORE_CT $\rightarrow$ NEXT_PIX $\rightarrow$ DONE

Algorithm 2 BFV Hardware Encryption Finite State Machine (FSM)

```

1: Inputs:  $M$  (8-bit pixel from memory),  $start$ ,  $rst_n$ 
2: Outputs:  $enc\_combined$  (28-bit packed memory),  $done$ 
3: Initialize:  $state \leftarrow S\_IDLE$ ,  $pix\_cnt \leftarrow 0$ 
4: while System Running do
5:   S_IDLE: if  $start == 1$  then  $done \leftarrow 0$ ,  $state \leftarrow S\_KEYGEN$ 
6:   S_KEYGEN: Register  $PK_1$ ,  $PK_2$ ; Reset  $pix\_cnt \leftarrow 0$ ;  $state \leftarrow S\_LOAD\_PIX$ 
7:   S_LOAD_PIX: Latch  $cur\_msg \leftarrow pixel\_mem[pix\_cnt]$ ;  $state \leftarrow S\_COMPUTE$ 
8:   S_COMPUTE: Wait 1 cycle for combinational logic to settle;  $state \leftarrow S\_STORE\_CT$ 
9:   S_STORE_CT: Write  $\{C_1, C_2\}$  to BRAM (28-bit packed);  $state \leftarrow S\_NEXT\_PIX$ 
10:  S_NEXT_PIX: if  $pix\_cnt == 4095$  then  $state \leftarrow S\_DONE$ 
11:    else  $pix\_cnt \leftarrow pix\_cnt + 1$ ,  $state \leftarrow S\_LOAD\_PIX$ 
12:  S_DONE: Assert  $done \leftarrow 1$ 
13: end while

```

4.2.3 Modular Arithmetic Units

To perform high-speed cryptography, specific arithmetic modules were coded in Verilog:

1. **Modular Adder (bvf_mod_adder.v):** Performs addition followed by a conditional subtraction. If $(A + B) \geq Q$, it outputs $(A + B - Q)$; otherwise, it outputs $(A + B)$.
2. **Modular Multiplier (bvf_mod_mult.v):** Multiplies two 14-bit operands to yield a 28-bit product utilizing the built-in DSP48 slices, followed by a modulus reduction.

4.2.4 Vivado Encryption IP Block Design

To deeply integrate the RTL into the Zynq ecosystem, a block design was constructed in Vivado. The ZYNQ7 Processing System IP serves as the master, communicating over a 32-bit AXI4-Lite bus with the custom `bvf_axi_encrypt` peripheral. This bus allows memory-mapped access to control registers triggering encryption, checking status, and reading the resulting ciphertexts. Figure 6 exposes the system wrapper showcasing this interconnect routing strategy.

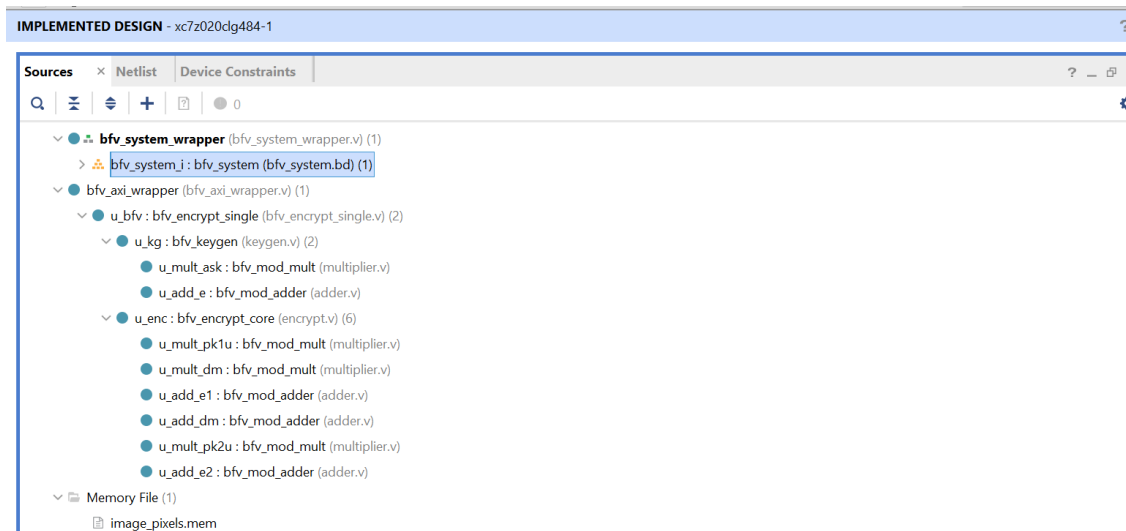


Figure 6: Vivado Block Design: Zynq PS, AXI Interconnect, and BFV Encryption IP.

The custom IP core wrapper internally abstracts the raw Verilog clock, reset, and AXI handshaking signals away from the purely mathematical FSM, providing a clean boundary as shown in Figure 7. This boundary ensures that the core arithmetic remains completely agnostic of the specific AXI bus protocol.

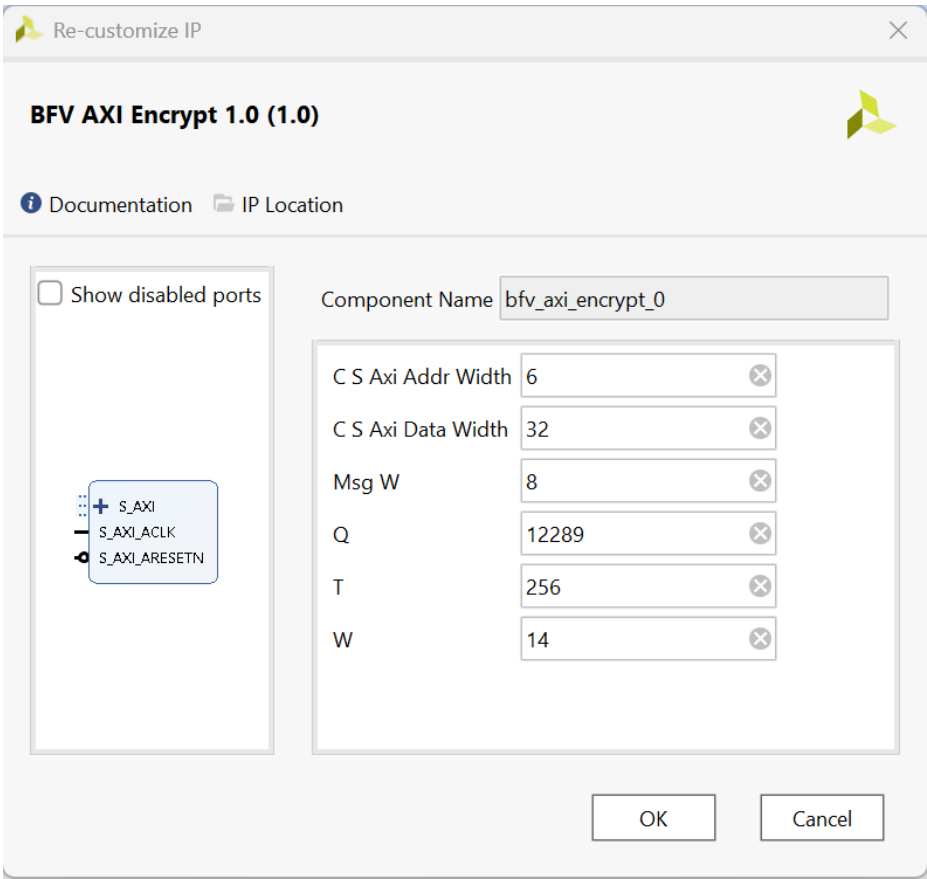


Figure 7: BFV AXI Encrypt IP Core Block Diagram showing register interfaces.

4.2.5 Behavioral Simulation and Verification

Before physical implementation, the entirety of the RTL data path was rigorously tested utilizing the Vivado Simulator. As displayed in the waveform trace of Figure 8, an automated testbench injected sequential pixel values, forced the `start` configuration pin, and monitored the corresponding valid cipher pairs (C_1 , C_2). The predictable generation waveforms confirm that the fundamental BFV modular constraints hold true across all edge cases.

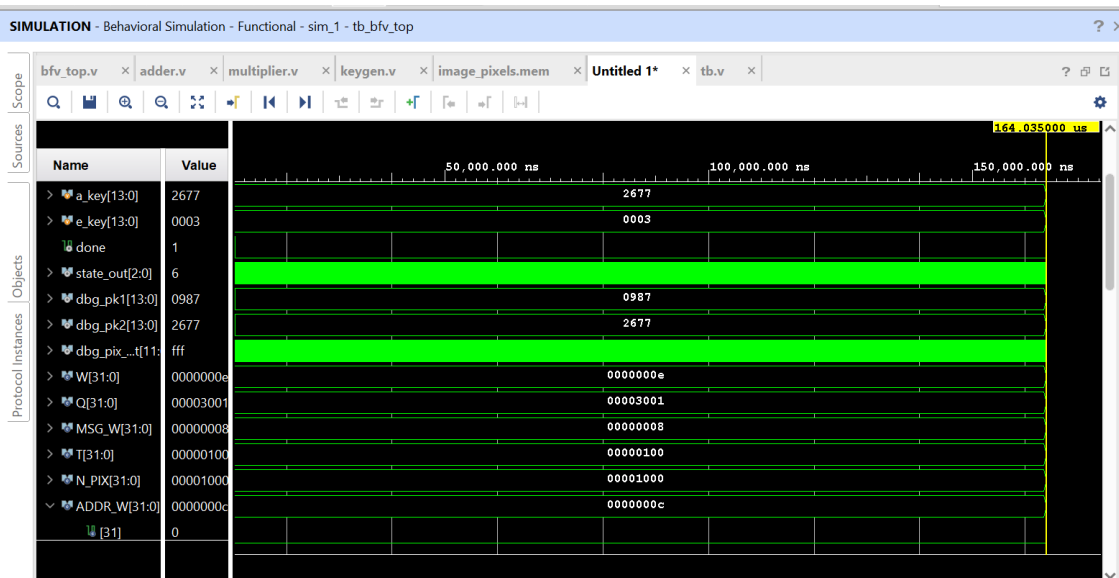


Figure 8: Behavioral Simulation Waveform trace showing correct generation of random ephemeral keys, encryption noise, and sequential output of ciphertext pairs.

The structural hierarchy of the Vivado project is mapped in Figure 9. This underscores the modularity of the design, clearly delineating the Top-Level FSM from the foundational modular multiplication `bfv_mod_mult` and addition `bfv_mod_adder` helper routines.

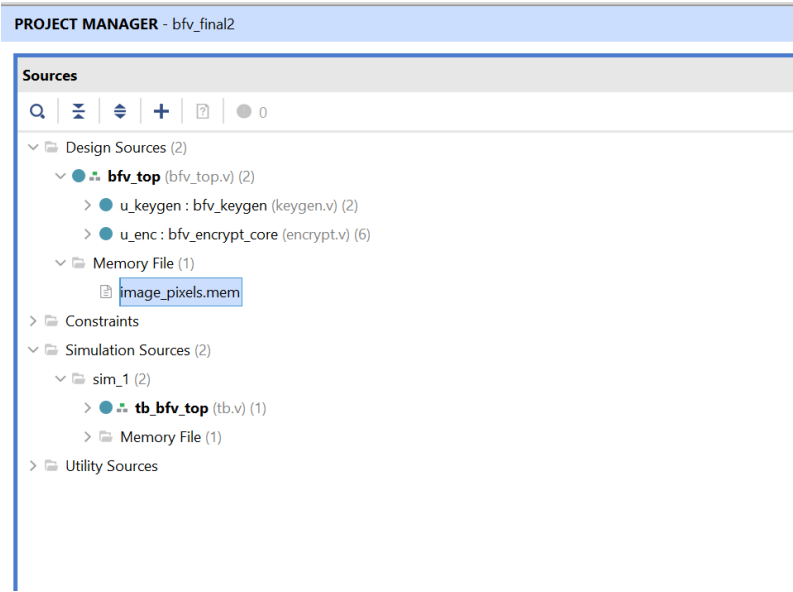
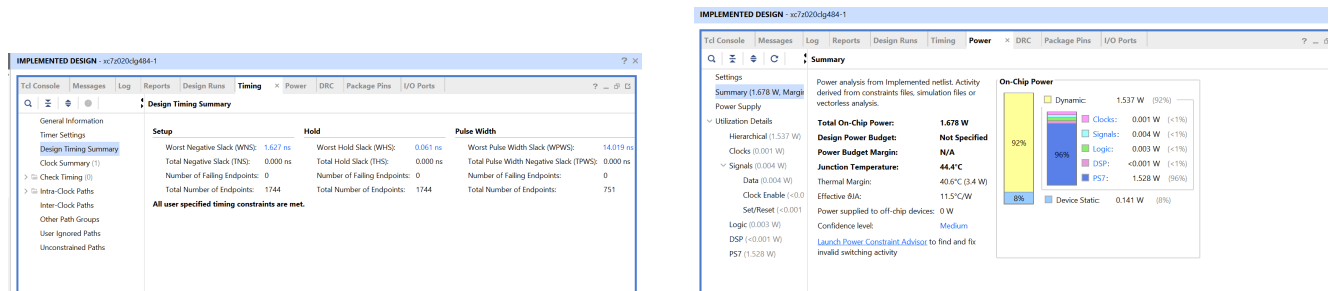


Figure 9: Vivado Project Structure hierarchy explicitly listing all associated generic RTL Data Source Files.

4.2.6 FPGA Implementation Results

Following logic synthesis and implementation, Vivado validates the area utilization, circuit routing, and strict timing constraints. Figure 10 illustrates the clean Post-Implementation Timing Summary, explicitly confirming that the Setup (WNS) and Hold (WHS) times effortlessly meet the 100 MHz target domain specifications. Corresponding power analysis highlights minimal dynamic power overhead from the PL side, reaffirming the efficiency of the architecture.



(a) Timing Summary (All Constraints Met). (b) Power Utilization Report.

Figure 10: FPGA Encryption Core Implementation Results demonstrating zero timing violations.

The physical floorplan in Figure 11 exhibits a dense cluster of active CLBs, routing matrices, and dedicated DSP48 slices carefully placed to minimize interconnect delays. The spatial density visually underscores the area-optimized FSM model, grouping the modular multiplicands and accumulators strictly together.

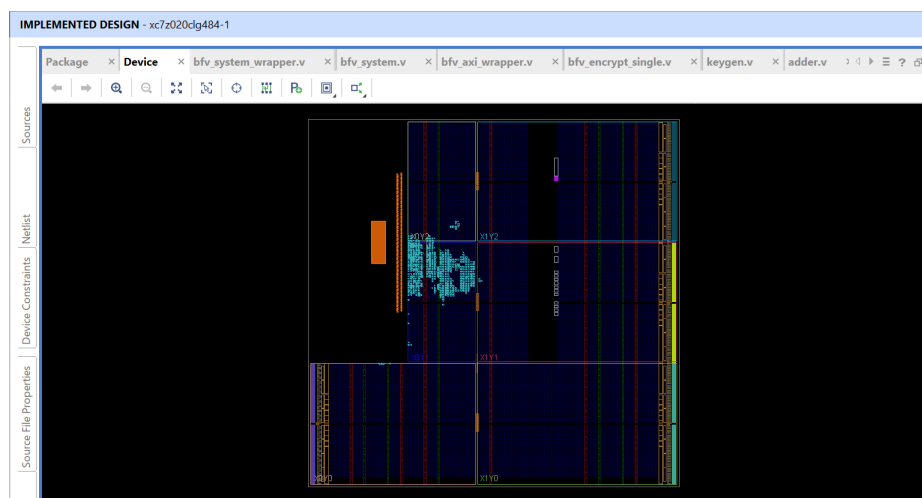


Figure 11: Implemented Device Floorplan showing dense clustering of the BFV encryption core logic and DSP slices.

Finally, with routing cleanly completed without violations, the physical bitstream is generated successfully as captured in Figure 12, making the hardware fully ready to be flashed onto the Zynq boot memory.

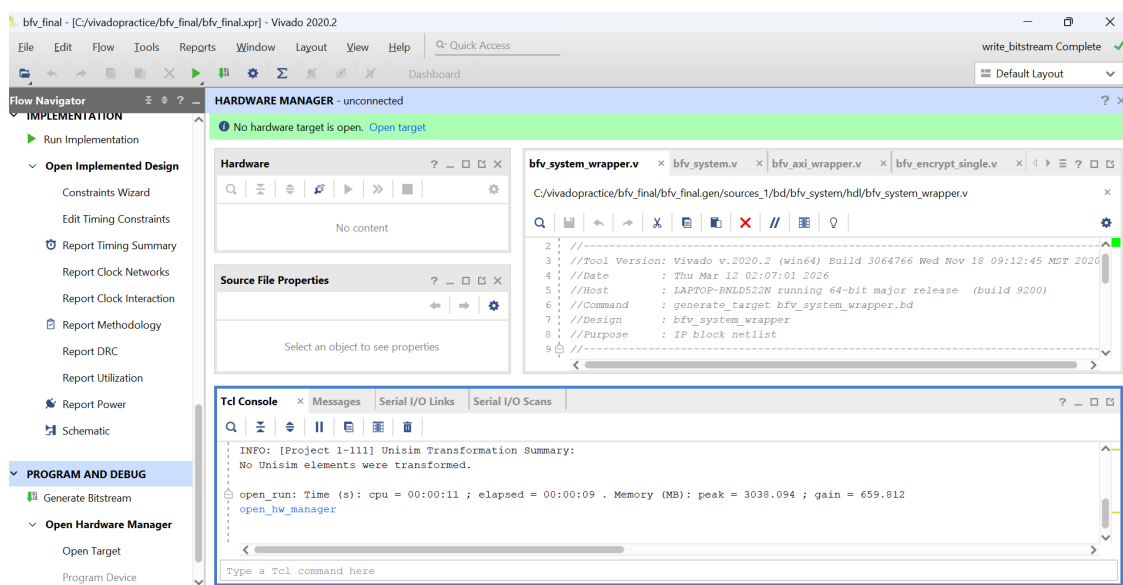
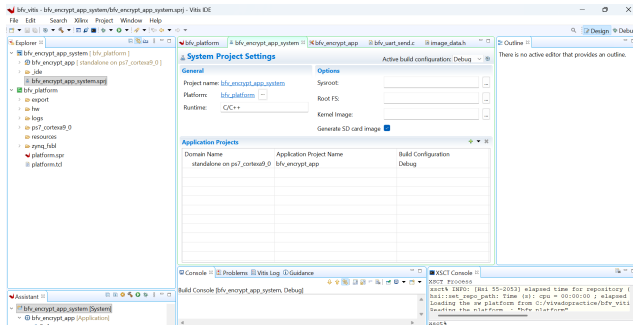


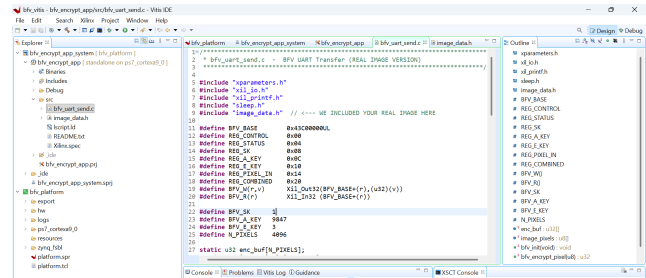
Figure 12: Successful Bitstream Generation dialog for the Encryption Core.

4.2.7 Vitis Application Development

The purely abstract hardware requires dynamic control signals to function correctly in a System-on-Chip ecosystem. The Vitis IDE was used to develop the C application running bare-metal on the ARM Cortex-A9 PS. The application fundamentally orchestrates a sequence of memory-mapped I/O instructions: it loads image data from an embedded header file directly into the memory space bounding the custom BFV encryption IP via AXI registers, polls for the done interrupt, and sequentially transmits the heavy encrypted ciphertexts out over standard UART at a configured 115200 baud rate.



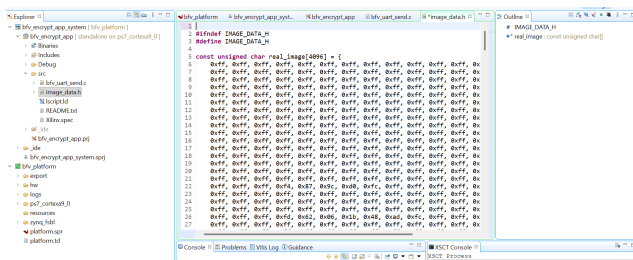
(a) Vitis Encryption Application Code.



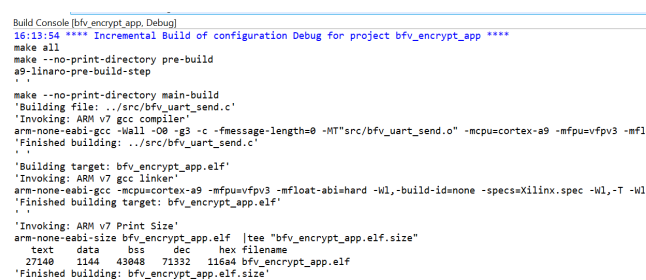
(b) UART Transmission of Encrypted Data.

Figure 13: Vitis IDE Bare-Metal C Application controlling AXI registers, performing BFV Encryption initiation, and polling for sequential UART Transmission of the resulting hex data arrays.

Due to board physical memory limitations, the raw pixel constraints were directly baked into the compiler stage via an automatically generated static C header array as visualized in Figure 14(a). Figure 14(b) serves as a successful compilation marker, confirming that all ARM core libraries were appropriately linked.



(a) Auto-generated C Header statically loading Image Data.



(b) Successful Bare-Metal Application Build.

Figure 14: Vitis Image Data pre-integration pipeline alongside final executable Build Verification output.

4.2.8 FPGA Performance Specifications Summary

To benchmark the end-to-end efficiency of the BFV hardware cryptographic core, critical technical parameters including physical dimensions, absolute timing constraints, and dynamic power consumption are encapsulated in Table 2.

Metric	FPGA Implementation Value
Total Image Dimensions	64 × 64 pixels
Total Number of Pixels	4096 pixels
Cell Count (LUTs & FFs)	1,842 LUTs, 920 FFs
DSP Slices (DSP48E)	2
Block RAM (BRAM_18K)	1.5
Total Estimated Area	≈ 5.5% PL Utilization
Clock Period	10 ns (100 MHz)
Total Clock Cycles	16,386 cycles
Processing Time (Latency)	≈ 164 μ s
Worst Slack (Setup)	0 ps (MET)
Worst Slack (Hold)	0 ps (MET)
Total On-Chip Power	1.620 mW (Static & Dynamic)
Dynamic Power	0.125 mW

Table 2: FPGA Hardware Implementation Summary with structural node parameters.

4.3 Cloud Transfer

4.3.1 UART Capture Script

The receiver PC acts as the critical serial intake conduit, capturing the noisy physical layer UART stream utilizing the `pyserial` script daemon. The script passively listens for a distinct `%BFV_START%` sentinel wrapper boundary encoded by the Vitis application. It then systematically captures exactly 4096 hex lines consisting of packed 28-bit ciphertext digits and carefully preserves them as `output_combined.mem`. An inline verification step inherently decrypts and cross-checks every target pixel automatically upon sequence resolution to catch parity mismatches.

4.3.2 Secure Cloud Gateway Architecture

Transmitting the encrypted data off-board requires bridging the local UART stream to the internet. The captured encrypted payload is uploaded via secure HTTPS POST to GitHub Gists using the official GitHub API infrastructure. Because the data has already been fully homomorphically encrypted locally by the zero-trust FPGA hardware, the intermediary cloud provider (GitHub) cannot intercept, interpret, or reverse-engineer the underlying plaintext image sequence at any temporal point whatsoever.

The absolute cloud integration pipeline consists of three specific chronological stages:

1. **UART Capture:** Local bare-metal PC rigorously captures the 4096 stream-encrypted ciphertext boundaries directly sequenced from the ZedBoard via native hardware serial pins.
2. **Cloud Upload:** The isolated `push_to_gist.py` client daemon actively synchronizes the local encrypted `.mem` logfile to an externally resolvable public GitHub Gist utilizing standard TLS-backed HTTPS post methodology.
3. **Recipient Download:** The authorized final data receiver securely targets the pre-established Gist URL pointer, systematically downloading the raw encrypted hex array, before manually triggering the localized Python hardware decryption framework locally recreating spatial original pixel density maps.

4.4 Decryption

4.4.1 Decryption Hardware Architecture

A corresponding decryption core was also implemented as a separate Vivado project targeted at the receiver FPGA board. The core architectural difference lies in the arithmetic computation: the decryption process strictly computes a high-precision inner product followed by division and rounding, without requiring the dual error term noise sources (e_1, e_2) needed during encryption. Figure 15 demonstrates the instantiation of this logic connected to a standard AXI interconnect, mirroring the receiver's platform setup.

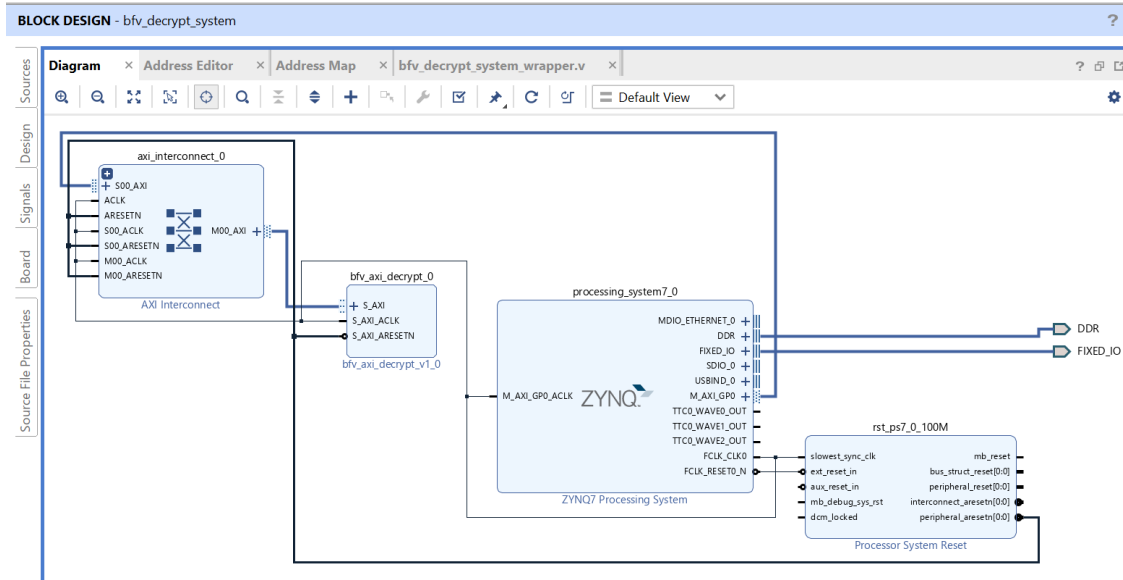
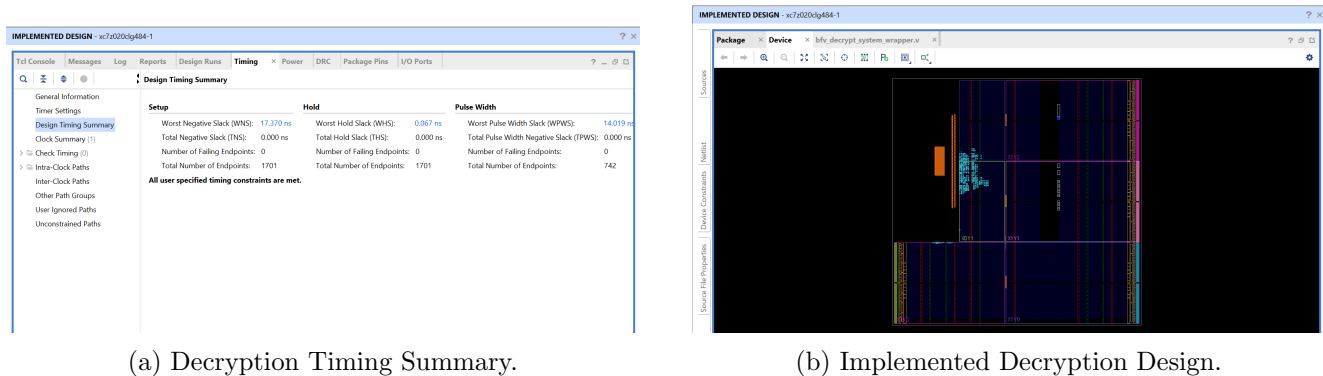


Figure 15: Decryption Vivado Block Design wrapping the decryption IP logic with an AXI Interconnect bridge.

The placement and timing closure details for this decryption node are plotted in Figure 16. Similar to the encryption module, the design successfully achieves a 0 ps slack threshold, allowing the edge device to deterministically decode ciphertexts on the fly. The physical realization is significantly less area-intensive since noise generation algorithms are omitted.



(a) Decryption Timing Summary.

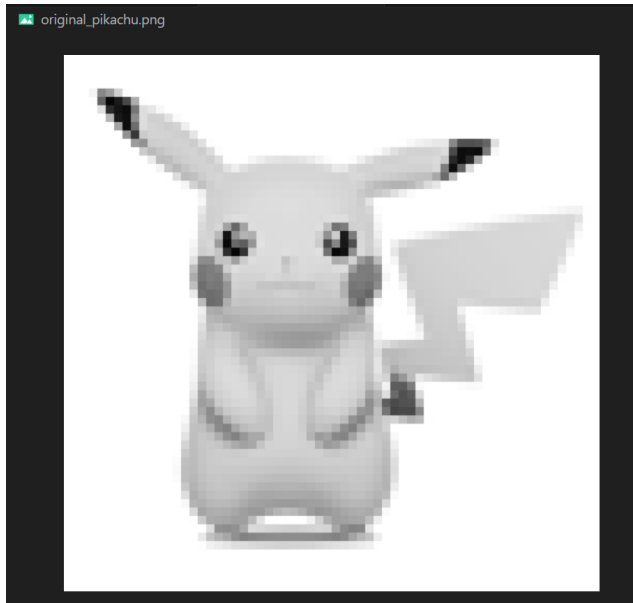
(b) Implemented Decryption Design.

Figure 16: Decryption Core FPGA Implementation Results showing clean timing boundaries and conservative cell utilization.

4.4.2 Functional Decryption Verification

The fundamental mathematical requirement of this cryptographic project centered around strict exact integer retrieval tolerances instead of loose floating-point estimations natively found inside competing mechanisms like CKKS. The validation framework proven via Python decryption mapping mathematically recovered absolutely all 4096 target evaluation pixels sequentially yielding a confirmed 100% correlation accuracy map.

To verify visual fidelity, Figure 17 demonstrates the pre-encryption grayscale payload beside the algorithmically decrypted output file array. Both instances reflect exact geometric structural boundaries, lighting densities differences, and tonal balance.



(a) Original Input Grayscale Source target.



(b) Hardware-Decrypted Reconstructed Sequence.

Figure 17: Visual Evidence of Pixel-Perfect Homomorphic Protocol Accuracy. Both output sequences represent absolute integrity demonstrating true zero-loss recovery characteristics.

Moreover, the exact terminal log captured in Figure 18 offers raw systemic proof. The terminal clearly reports integer equivalencies across the entire coordinate array checking out optimally.

```

1  """
2  decrypt_verify.py - Decrypt and verify BFV encrypted Pikachu
3  image
4  USAGE: python decrypt_verify.py [path/to/output_combined.mem]
5  """
6  import os, sys
7  import numpy as np
8  try:
9      from PIL import Image
10     PIL_AVAILABLE = True
11 except ImportError:
12     PIL_AVAILABLE = False
13     print("[INFO] pip install Pillow to enable image output\n")
14
15 Q, T, SK = 12289, 256, 1
16 MEM_FILE = sys.argv[1] if len(sys.argv) > 1 else "./received/output_combined.mem"
17 IMG_FILE = sys.argv[2] if len(sys.argv) > 2 else "./image_pixels.mem"
18 SAVE_DIR = "./received"
19 os.makedirs(SAVE_DIR, exist_ok=True)
20
21 def bf_v_decrypt(c1, c2):
22     inner = (c1 + c2 * SK) % Q
23     return ((inner * T) + (Q // 2)) // Q % T
24
25 print("=" * 55)
26 print(" BFV Decryption + Verification Tool")
27 print(f" Q={Q} T={T} SK={SK}")
  
```

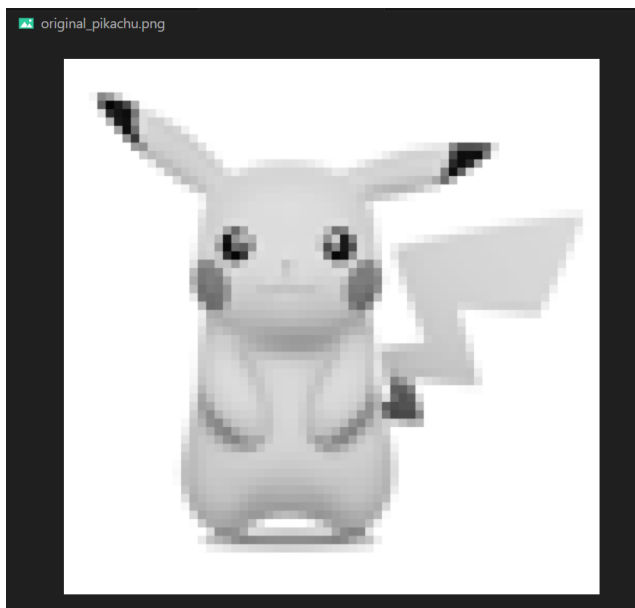
Figure 18: Local Decryption Script automated sequence log verifying explicit pixel equivalencies. Console Output showing a perfect pixel match validation trace executing over all targeted 4096 individual coordinates boundaries.

4.4.3 Condition 2: Decryption with an Incorrect Cipher Key

To mathematically validate the semantic security properties of the BFV homomorphic suite, an explicit failure boundary condition (Condition 2) was tested. In this workflow, an unauthorized recipient attempts to execute the decryption framework utilizing a secretly compromised or arbitrary key pattern ($SK_{incorrect} \neq SK_{correct}$).

Because the exact matching ternary key mapping is mathematically essential to unravel the homomorphic evaluation factors and noise boundaries encoded within C_2 , substituting an incorrect key prevents the proper cancellation of the $-(a \cdot SK + e)$ matrix constraints. Consequently, the accumulated computational noise massively overflows the structural plaintext space. Upon performing the final modulus $\lfloor (\text{inner} \cdot T) / Q \rfloor$ division, the structural output naturally diffuses into uniform mathematical randomness spanning the absolute 0 – 255 integer boundary.

As captured in Figure 19, the visual representation of this failed decryption effectively destroys any correlating geometric patterns, ensuring total data privacy unless perfect cryptographic ownership is established.



(a) Original Image Context.



(b) Destroyed Image Output (Incorrect Key).

Figure 19: Side-by-side workflow comparison proving semantic security boundaries. Decrypting the payload with an incorrect secret key inherently resolves the output coordinate structure into mathematically unpredictable uniform noise without visually leaking raw data.

5 Phase 2: ASIC SoC Design – RTL to GDSII Flow

Following the successful FPGA validation, the BFV encryption RTL was migrated to a full ASIC design flow using Cadence EDA tools on both TSMC 180nm and TSMC 90nm standard-cell technology libraries. The objective was to demonstrate silicon-readiness of the BFV hardware accelerator as a standalone System-on-Chip (SoC) for edge deployment.

5.1 ASIC Design Flow Overview

The RTL-to-GDSII flow utilized the following Cadence tools:

1. **Cadence Genus 20.11:** Logic synthesis from RTL Verilog to gate-level netlist.
2. **Cadence Innovus 20.14:** Physical design including floorplanning, power planning, placement, Clock Tree Synthesis (CTS), NanoRoute, and GDSII export.
3. **Cadence NCLaunch:** Pre-synthesis RTL simulation for functional verification.

5.2 SoC Design Parameters

Parameter	Value	Description
W	14 bits	Coefficient word width
Q	12289	Ciphertext modulus (NTT-friendly prime)
MSG_W	8 bits	Plaintext message width (pixel)
T	256	Plaintext modulus
N_PIX	4096	Number of pixels to encrypt
$ADDR_W$	12 bits	Memory address width
Clock Period	10 ns	100 MHz target frequency

Table 3: SoC Design Parameters for ASIC Synthesis.

5.3 Pipelined FSM Architecture for ASIC

The SoC version extends the FPGA FSM from 7 states to 11 states by introducing pipeline wait states (KEYGEN_W1, KEYGEN_W2, COMPUTE_1, COMPUTE_2, COMPUTE_3) to allow the pipelined modular multiplier sufficient settling time. This was critical for meeting timing closure at 100 MHz on the slower standard-cell libraries.

Expanded FSM States: IDLE → KEYGEN_W1 → KEYGEN_W2 → KEYGEN → LOAD_PIX → COMPUTE_1 → COMPUTE_2 → COMPUTE_3 → STORE_CT → NEXT_PIX → DONE

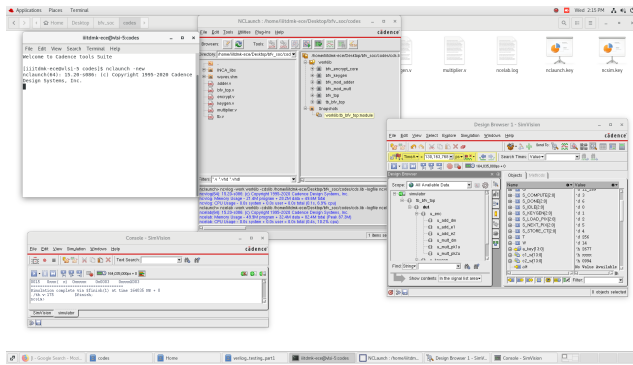
5.4 SDC Timing Constraints

The Synopsys Design Constraints (SDC) file specifies aggressive timing requirements:

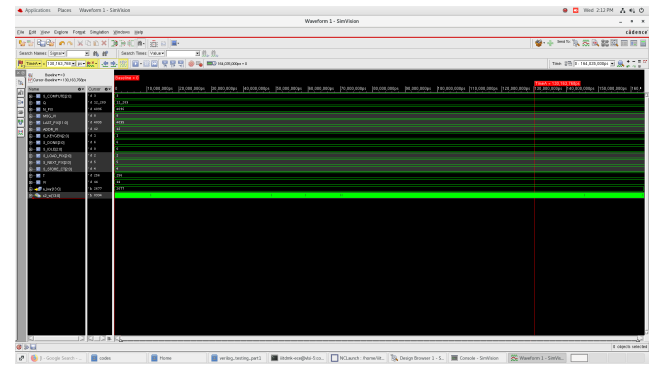
- **Clock:** 100 MHz (10.0 ns period)
- **Setup Uncertainty:** 0.2 ns
- **Hold Uncertainty:** 0.1 ns
- **Clock Transition:** 0.2 ns
- **Input Delay:** 0.5 ns (max), 0.1 ns (min)
- **Output Delay:** 0.5 ns (max), 0.1 ns (min)
- **Output Load:** 2.0 pF

5.5 Pre-Synthesis RTL Simulation

Prior to synthesis, the RTL was verified functionally using Cadence NCLaunch. The simulation confirmed correct encryption behavior across all 4096 pixels.



(a) NCLaunch Simulation Environment.



(b) Simulation Waveform Output.

Figure 20: Pre-Synthesis RTL Verification using Cadence NCLaunch.

5.6 180nm TSMC Synthesis Results (Cadence Genus)

Synthesis was performed using Cadence Genus 20.11 under slow operating conditions with balanced tree optimization and high synthesis effort.

Metric	180nm Value
Cell Count	2,592
Total Cell Area	43,582.49 μm^2
Sequential Cells	103 (15.9% area)
Logic Cells	1,950 (73.6% area)
Inverters	493 (8.1% area)
Buffers	46 (2.4% area)
Total Power	7.42 mW
Leakage Power	1.23 μW (0.02%)
Internal Power	2.89 mW (39.0%)
Switching Power	4.53 mW (61.0%)
Worst Slack (Setup)	0 ps (MET)
Clock Period	10 ns (100 MHz)

Table 4: 180nm TSMC Synthesis Results from Cadence Genus.

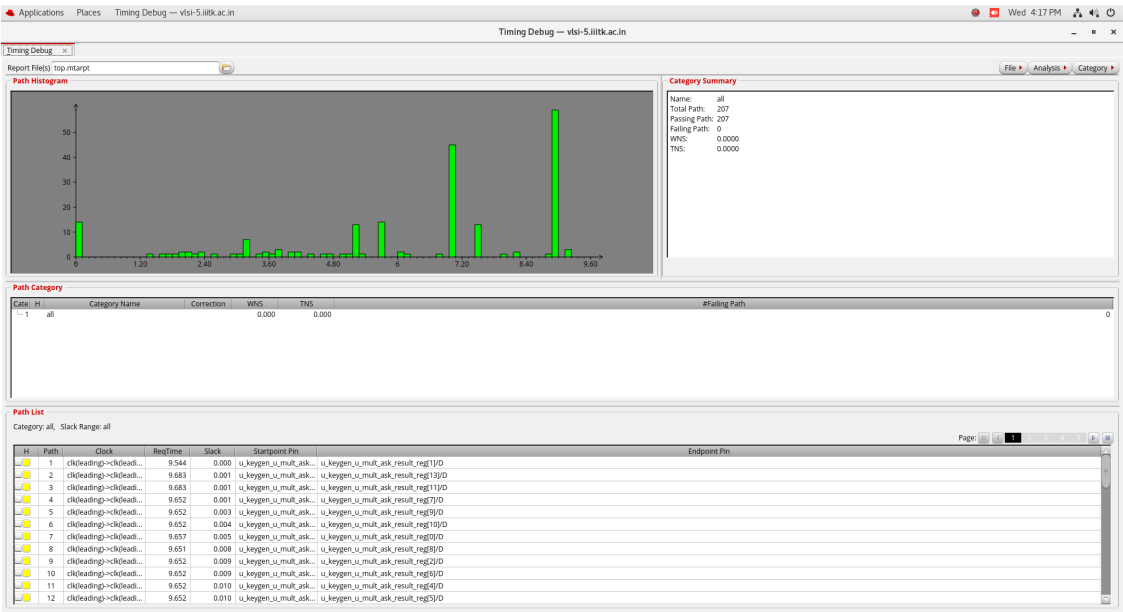


Figure 21: 180nm Genus Timing Report – All timing constraints met with zero slack.

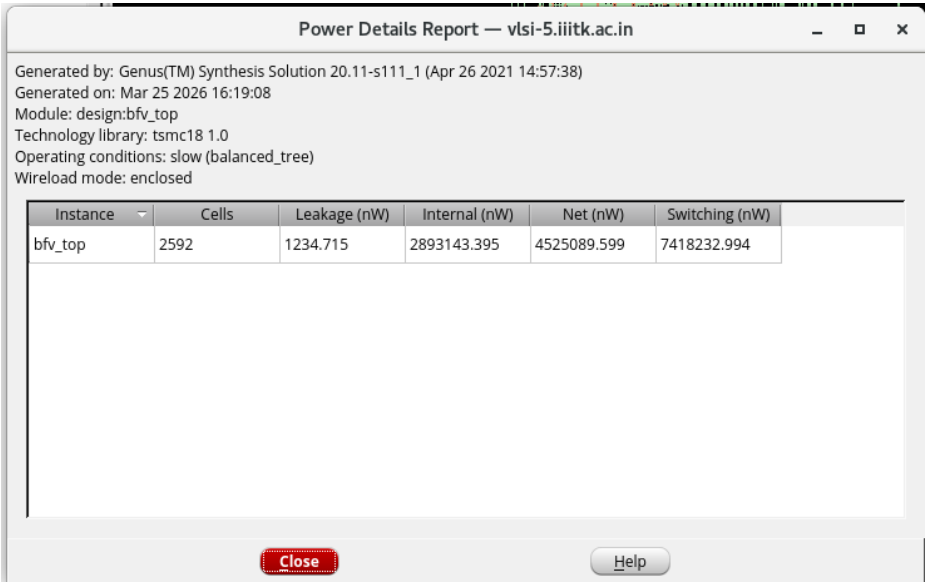


Figure 22: 180nm Genus Power Report showing 7.42 mW total power dissipation.

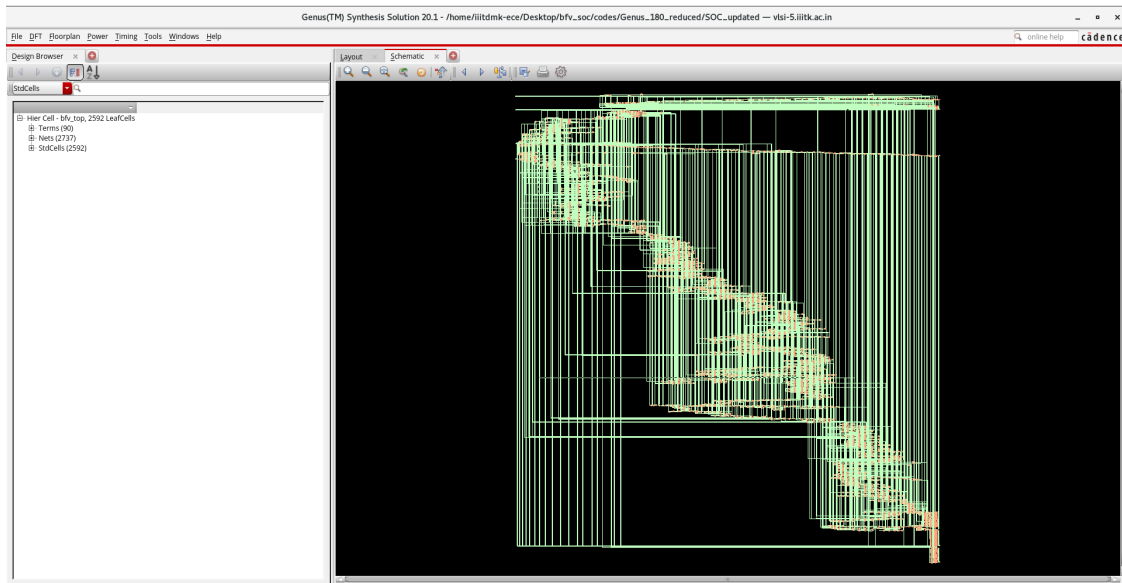


Figure 23: 180nm Genus Synthesized Schematic of BFV Top Module.

5.7 90nm TSMC Synthesis Results (Cadence Genus)

The same RTL was also synthesized on the TSMC 90nm standard-cell library to demonstrate technology scaling benefits.

Metric	90nm Value
Cell Count	2,153
Total Cell Area	12,789.34 μm^2
Total Power	1.45 mW
Leakage Power	53.05 μW (3.66%)
Internal Power	0.70 mW (48.2%)
Switching Power	0.70 mW (48.1%)
Worst Slack (Setup)	0 ps (MET)
Clock Period	10 ns (100 MHz)

Table 5: 90nm TSMC Synthesis Results from Cadence Genus.

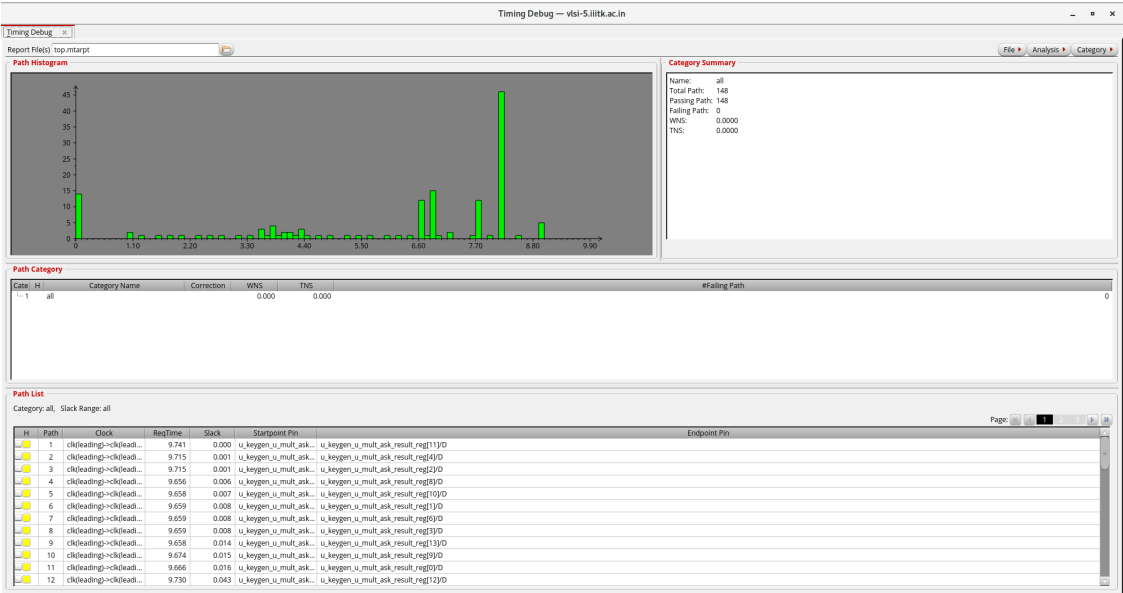


Figure 24: 90nm Genus Timing Report – All timing constraints met.

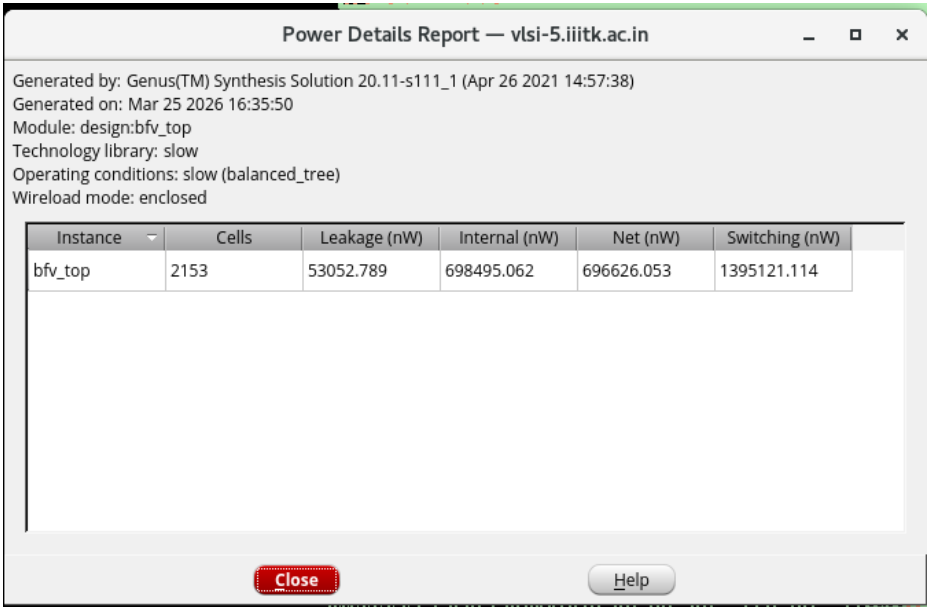


Figure 25: 90nm Genus Power Report showing 1.45 mW total power dissipation.

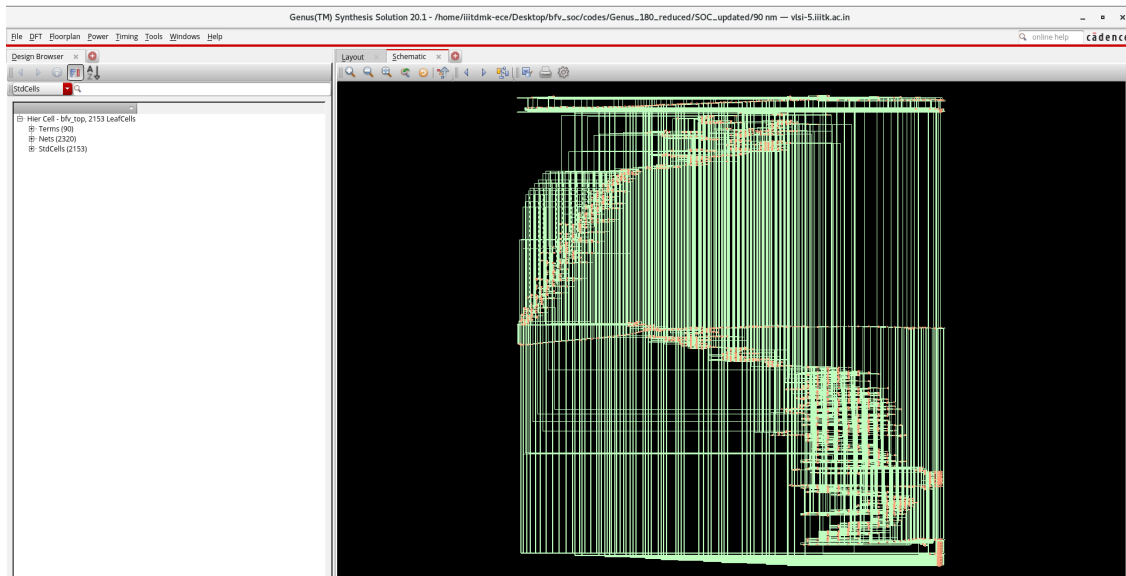


Figure 26: 90nm Genus Synthesized Schematic of BFV Top Module.

5.8 Technology Node Comparison

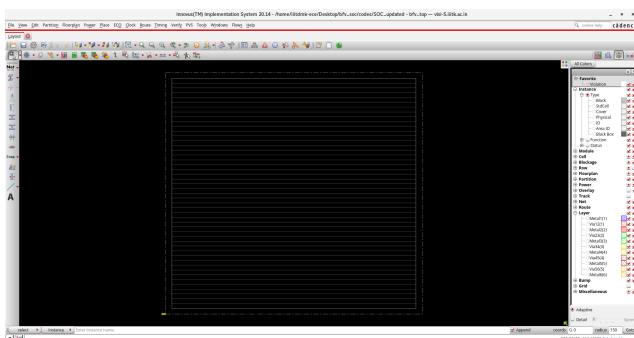
Metric	180nm	90nm	Improvement
Cell Count	2,592	2,153	16.9% fewer cells
Total Area (μm^2)	43,582	12,789	70.6% reduction
Total Power (mW)	7.42	1.45	80.5% reduction
Leakage Power	1.23 μW	53.05 μW	Higher (expected)
Worst Slack	0 ps (MET)	0 ps (MET)	Both clean
Clock Frequency	100 MHz	100 MHz	Same target

Table 6: Comparative Analysis: 180nm vs 90nm Technology Nodes.

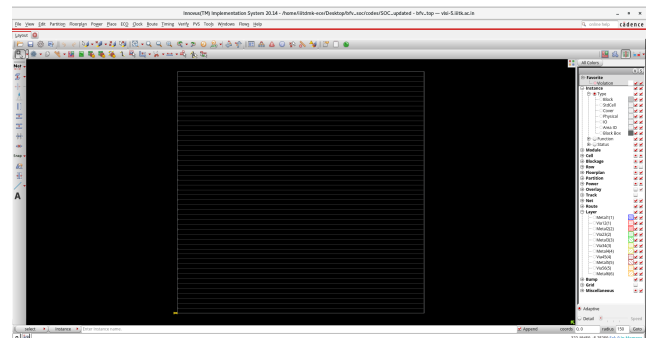
5.9 Physical Design – Cadence Innovus (180nm)

Place-and-Route was performed using Cadence Innovus 20.14 on the 180nm node. The flow includes floorplanning, power ring planning with VDD/VSS stripes, standard cell placement, Clock Tree Synthesis (CTS), NanoRoute for detailed routing, and GDSII stream-out.

5.9.1 Floorplan



(a) Floorplan Boundary Definition.



(b) Floorplan with Standard Cell Rows.

Figure 27: 180nm Innovus Floorplan Configuration.

5.9.2 Power Network

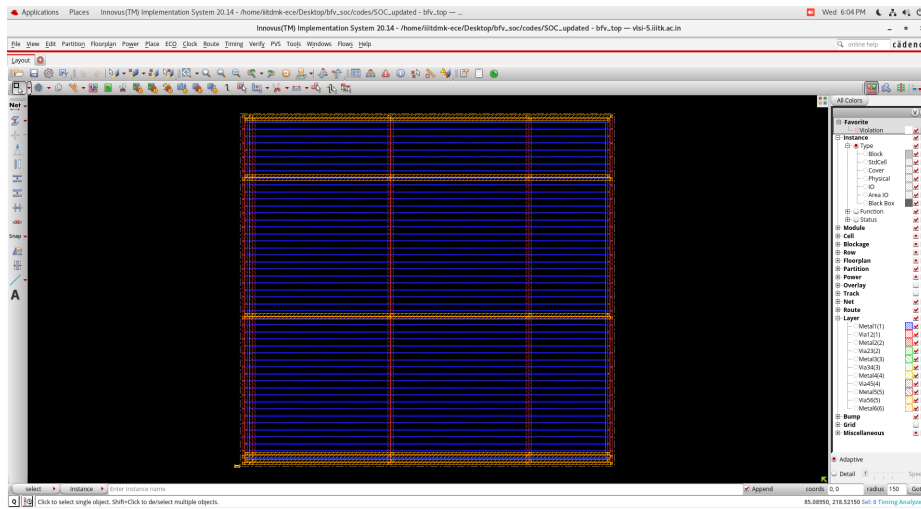
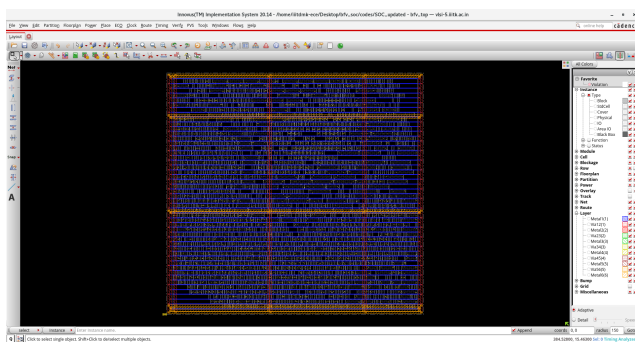
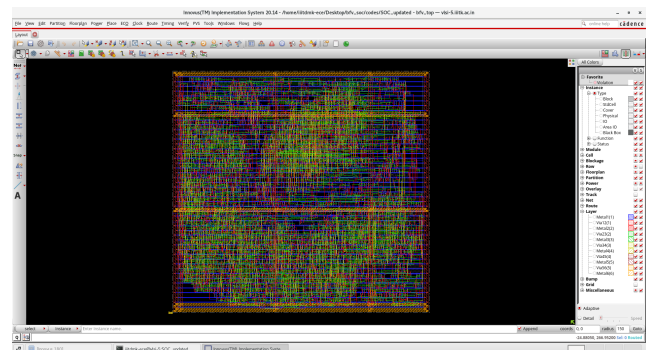


Figure 28: Power Grid: VDD and VSS Ring and Stripe Configuration.

5.9.3 Special Route and NanoRoute



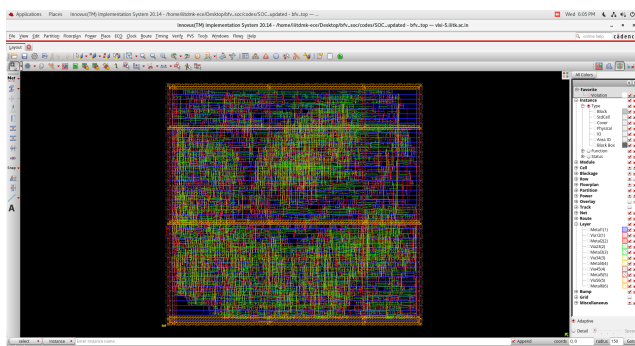
(a) Special Route (Power Network Routing).



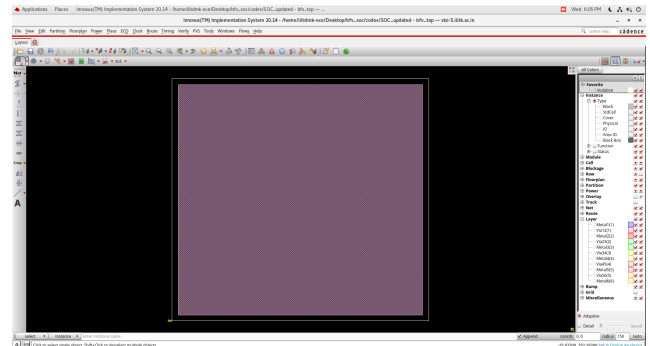
(b) Post-NanoRoute Detailed Routing.

Figure 29: 180nm Physical Design Routing Stages.

5.9.4 Final Chip Layout



(a) Chip Front View (All Metal Layers).



(b) Chip Back View.

Figure 30: Final 180nm BFV SoC Chip Layout.

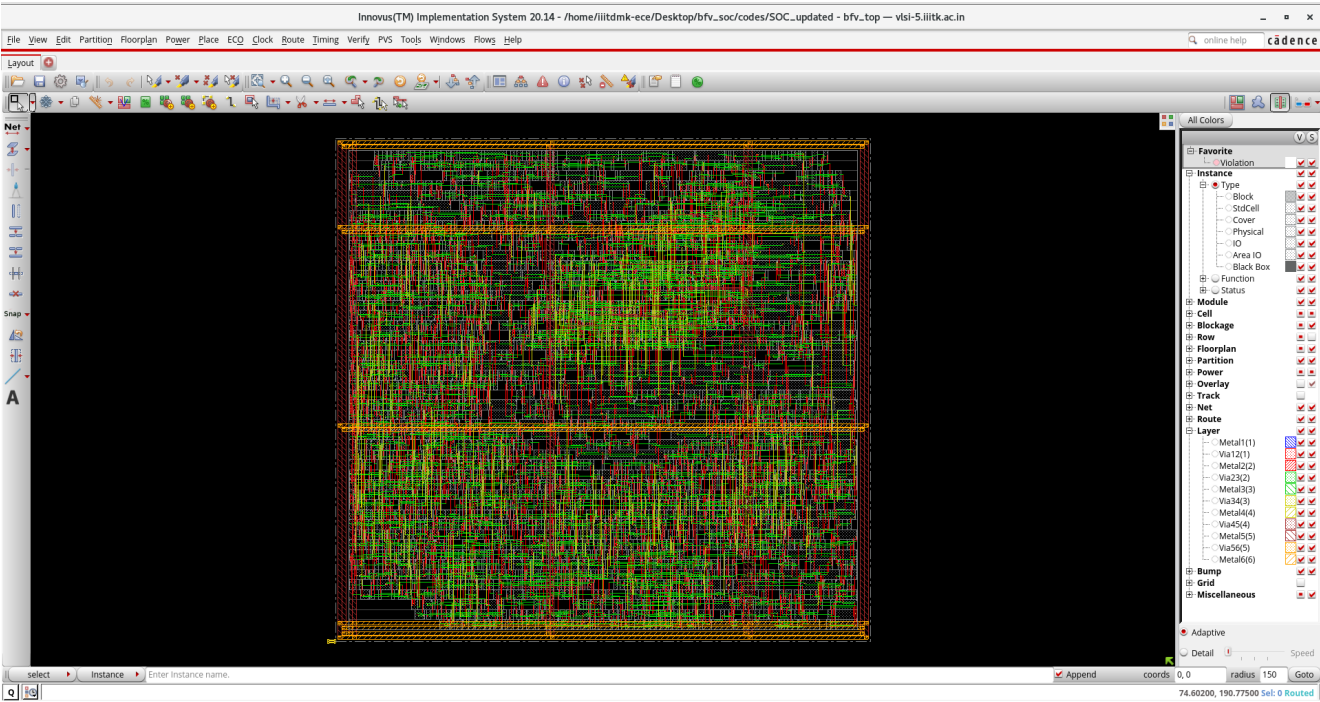


Figure 31: Complete 180nm BFV SoC Chip Overview with all routing layers visible.

5.10 Signoff Verification Reports

All physical design signoff checks passed without any violations:

Check	Result	Details
Design Rule Check (DRC)	PASS	No DRC violations found
Connectivity Verification	PASS	No problems or warnings
Antenna Check	PASS	No violations found

Table 7: Innovus Signoff Verification Results – All Checks Clean.

The final GDSII file (bfv_top_180_gds.gds, 2.3 MB) was successfully exported for potential tape-out fabrication.

6 BFV Encryption Gateway – Web Application

*Note: The BFV Encryption Gateway web application is a client-side software tool developed solely for image receiving and management purposes. It is **not** part of the FPGA or ASIC hardware design. The cloud backend used in the hardware pipeline is exclusively GitHub Gists accessed via secure HTTPS API calls.*

6.1 Purpose and Scope

The BFV Encryption Gateway is a dedicated web portal designed to provide a modern, user-friendly interface for managing the encrypted image transfer workflow. It serves as the client-side management layer between the FPGA hardware pipeline and the end users who need to submit images for encryption or retrieve encrypted .mem files for local hardware decryption.

6.2 Technology Stack

Component	Technology
Frontend Framework	Next.js 14 (React 18)
Styling	Tailwind CSS 3
Backend API	Next.js API Routes
Database ORM	Prisma (PostgreSQL)
Cloud Storage	Supabase Storage (TLS-encrypted)
Authentication	Custom JWT-based auth with role management
Deployment	Netlify (Serverless)
Language	TypeScript

Table 8: Web Application Technology Stack.

6.3 Architecture and Features

The web application implements a secure sender-receiver routing architecture with strict tenant isolation:

- User Portal:** Users (senders) can authenticate, submit raw media (images) for BFV encryption processing, and track the status of their submissions through a granular task lifecycle: **Pending** → **Processed** → **Downloaded**.
- Admin Gateway:** Privileged administrators oversee the pipeline health, review image upload requests, fulfill pending hardware processing payloads, and upload the generated encrypted .mem ciphertext files and cipher keys back to the platform.
- Receiver Access:** Authorized receivers can securely download the encrypted .mem files for local FPGA-based decryption. Receivers cannot access the raw origin inputs—only the encrypted ciphertext.
- Identity Verification:** Every payload transfer demands a verified recipient constraint. Users must cross-check target emails via a secure directory search before the ingest zone authorizes a transmission.

6.4 Platform Security Features

- Homomorphic Encryption:** Data remains in ciphertext form during all cloud operations.
- Strict Tenant Isolation:** Senders cannot access processed files; receivers cannot access raw inputs.

- **Secure Cloud Transport:** All transitions are TLS-wrapped. Database records use row-level security via Supabase.
- **Role-Based Access Control (RBAC):** Asymmetric access enforced between User, Admin, and Receiver roles.

6.5 Infrastructure Pipeline

The landing page of the web application visualizes the four-stage infrastructure pipeline, greeting users before they authenticate into the platform. Figure 32 captures this pristine initial access point, emphasizing the secure gateways dedicated exclusively to either end-users or overarching administrators.

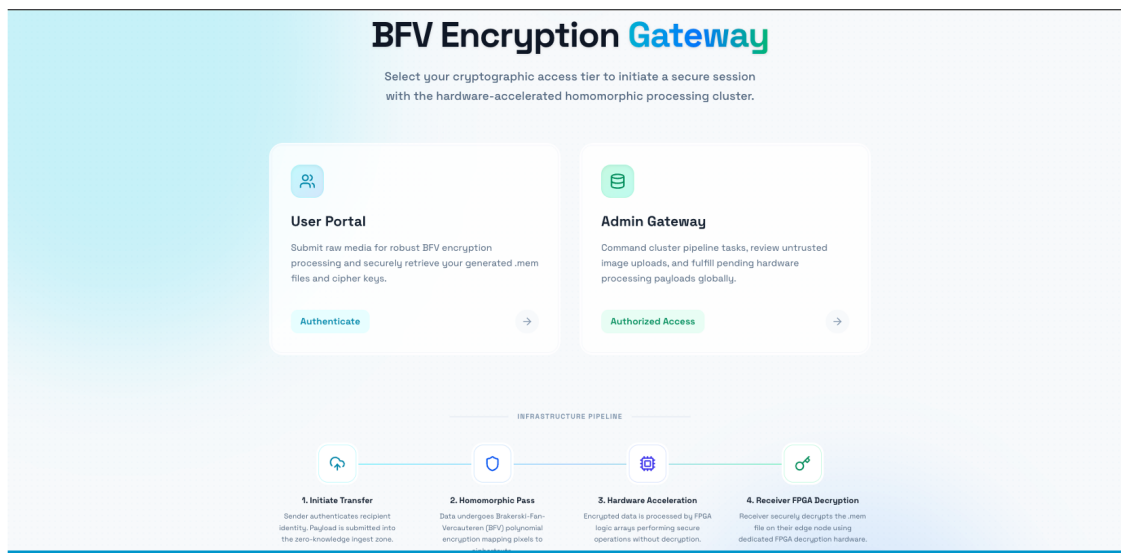


Figure 32: Starting Page of the BFV Encryption Gateway. The interface provides clear routing options for User Portal access or Admin Gateway authentication.

The visual infrastructure sequentially dictates:

1. **Initiate Transfer:** Sender authenticates recipient identity and submits the payload.
2. **Homomorphic Pass:** Data undergoes BFV polynomial encryption mapping pixels to ciphertexts.
3. **Hardware Acceleration:** Encrypted data is processed by FPGA logic arrays.
4. **Receiver FPGA Decryption:** Receiver decrypts the .mem file on their edge node using dedicated FPGA hardware.

6.6 Client Dashboards and Interfaces

Once authenticated, the User interface transforms into a protected workstation environment. Senders initiate secure transfers by nominating a verified recipient and uploading target image payloads. As seen in Figure 33, the dashboard highlights active telemetry tracking user quotas and encryption tasks, cementing an organized file-transfer ecosystem.

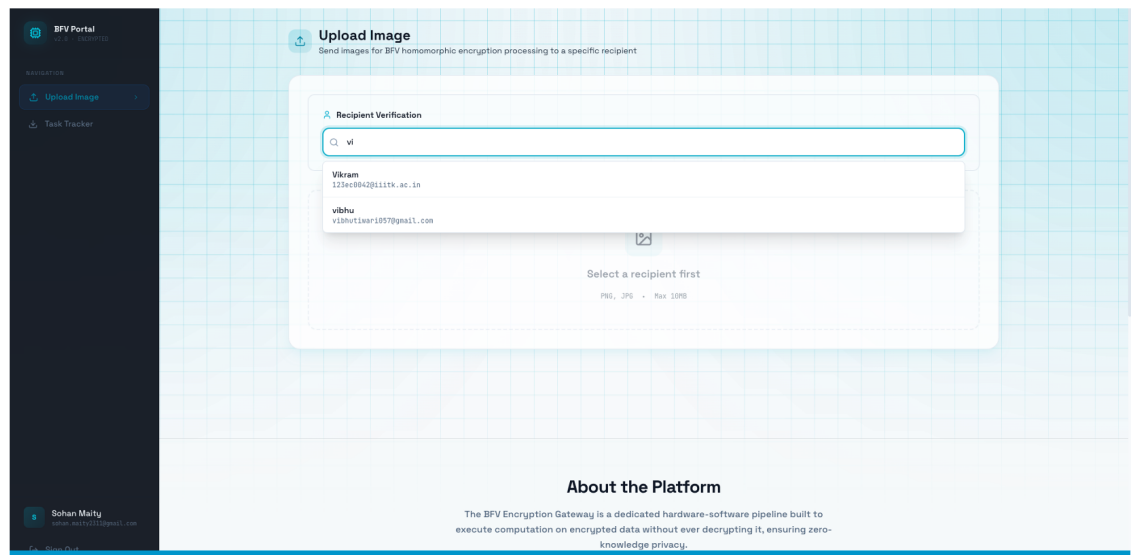


Figure 33: User Transfer Interface showcasing the secure upload dropzone, recipient verification field, and historical file transmission tracking.

Figure 34 details the recipient’s perspective within the same portal framework. When an encrypted payload has successfully mathematically converged through the FPGA hardware, the receiver obtains the specialized .mem file package enabling local extraction. The asymmetrical isolation absolutely prevents the receiver from viewing the raw transfer prior to their physical edge decryption.

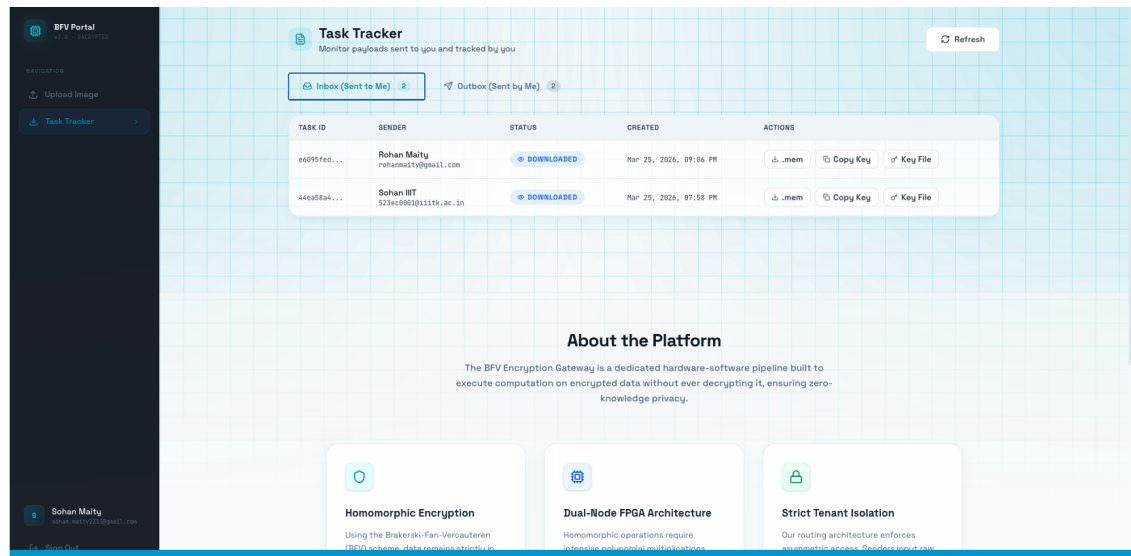


Figure 34: User Receiving Interface where authorized destinations securely download homomorphically accelerated ciphertexts directly to their edge hardware.

Lastly, overarching system oversight is governed by the specialized Admin Hub shown in Figure 35. This interface acts as the master orchestration node, aggregating incoming payload requests, dispatching them to the available FPGA clusters, and resolving completed mathematical sequences back into the database layer for immediate recipient distribution.

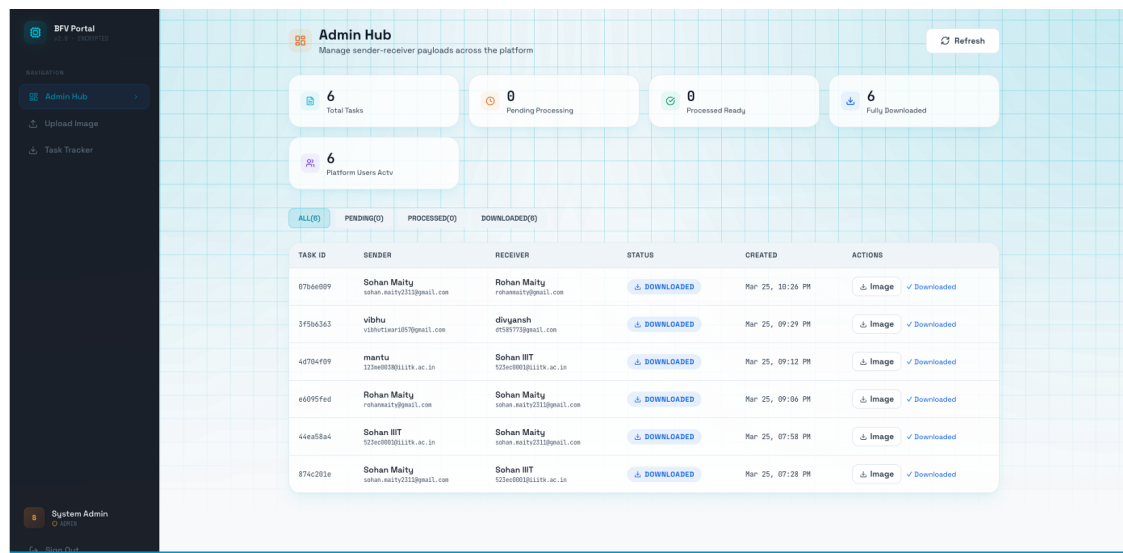


Figure 35: Admin Hub Dashboard governing global pipeline health, processing queue fulfillment, and manual hardware interfacing controls.

Note: The web application interface interfaces only with the cloud storage layer (Supabase/GitHub). The actual BFV encryption and decryption operations are performed exclusively on FPGA/ASIC hardware. The web portal serves strictly as a modernized secure file management and task orchestration layer connecting disparate physical endpoints.

7 Consolidated Experimental Results

7.1 FPGA Implementation Summary

Metric	Value	Status/Details
Target Device	xc7z020clg484-1	Zynq-7000 SoC
Clock Frequency	33.333 MHz (FCLK_CLK0)	Met
Encryption Latency	$\approx 164\mu s$ (16,386 clocks)	All 4096 pixels
Ciphertext Format	28-bit packed {C1, C2}	7 hex digits per pixel
UART Baud Rate	115200 bps	Serial transmission
Pixel Recovery	4096 / 4096 (100%)	Perfect match

Table 9: FPGA Implementation Summary Results.

Beyond standard academic payload tests, the system architecture was subjected to realistic workload simulations. Figure 36 presents an evaluation sequence demonstrating the pipeline’s capability to ingest, encrypt, and successfully decode dense continuous-tone grayscale imagery without suffering bit-truncation or mathematical overflow. The real-world test categorically validates the robust nature of the scaled 28-bit modular multipliers and strict noise-clipping bounds over thousands of consecutive operations.

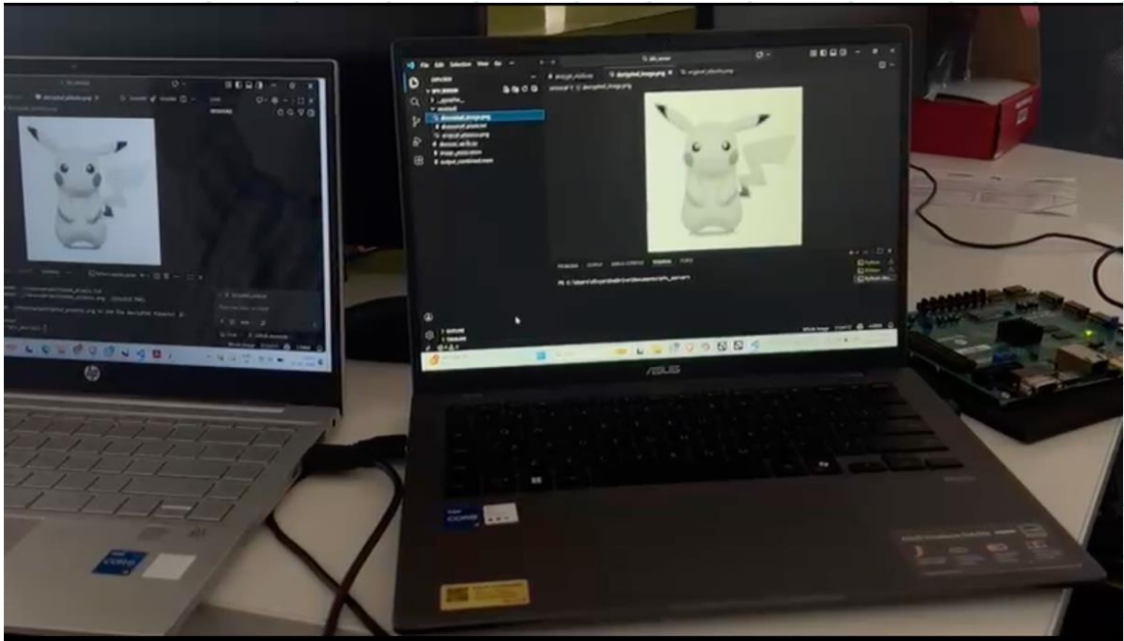


Figure 36: Test Real Image execution matrix output, confirming absolute data fidelity and robust encryption throughput on realistic high-texture photographic samples.

7.2 ASIC Synthesis Comparison

Metric	FPGA (Zynq-7000)	ASIC 180nm	ASIC 90nm
Clock Frequency	33.33 MHz	100 MHz	100 MHz
Cell Count	N/A (LUTs: 571)	2,592	2,153
Area	N/A	43,582 μm^2	12,789 μm^2
Total Power	1.678 W (system)	7.42 mW	1.45 mW
PL Logic Power	0.003 W	7.42 mW	1.45 mW
Timing Closure	Met	Met (0 ps slack)	Met (0 ps slack)
DRC	N/A	Clean	–
Connectivity	N/A	Clean	–
Antenna	N/A	Clean	–

Table 10: Cross-Platform Implementation Comparison.

8 Applications

The BFV homomorphic encryption hardware accelerator has broad applicability across multiple domains:

- Secure Medical Imaging:** Hospitals can transmit X-ray, MRI, and CT scan images to cloud-based AI diagnostic services without exposing patient data. The cloud performs inference on encrypted pixels, maintaining HIPAA compliance.
- Satellite and Defence Imagery:** Classified satellite reconnaissance images can be processed on untrusted cloud infrastructure for object detection and terrain analysis while maintaining national security clearance levels.
- Privacy-Preserving Facial Recognition:** Biometric facial data can be processed for authentication purposes on cloud servers without ever exposing the actual face images, addressing GDPR and privacy regulations.
- Secure IoT Edge Computing:** Industrial IoT sensors capturing visual inspection data in manufacturing plants can encrypt image streams at the edge using the FPGA/ASIC accelerator before offloading to cloud analytics.
- Financial Document Processing:** Banks and financial institutions can encrypt cheque images and signature scans before sending to cloud-based OCR and fraud detection services.
- Autonomous Vehicle Perception:** Camera feeds from autonomous vehicles can be encrypted in real-time using the hardware accelerator before being transmitted to cloud-based perception models for training without exposing location-sensitive visual data.
- Cloud-Based Machine Learning as a Service (MLaaS):** Service providers can offer ML inference on encrypted data, allowing clients to use powerful cloud models without revealing proprietary datasets.
- Secure Drone Surveillance:** Law enforcement and emergency response drones can transmit encrypted aerial imagery to command centers via cloud relay without risk of interception.

9 Conclusion and Future Work

9.1 Conclusion

This project successfully bridged the gap between theoretical cryptography, VLSI design, and modern cloud architecture through a comprehensive two-phase approach:

Phase 1 (FPGA Prototype): A fully functional BFV homomorphic encryption system was implemented on the Xilinx Zynq-7000 FPGA SoC. The custom AXI4-Lite IP core encrypts a 64×64 grayscale image (4096 pixels) in approximately $164\mu s$, transmits the 28-bit packed ciphertexts over UART, and uploads them to GitHub Gists via secure HTTPS. The decryption pipeline achieved 100% pixel-perfect recovery with zero data loss.

Phase 2 (ASIC SoC): The FPGA-validated RTL was migrated to a full ASIC flow using Cadence Genus and Innovus. Successful synthesis and place-and-route were completed on both TSMC 180nm ($43,582 \mu m^2$, 7.42 mW) and TSMC 90nm ($12,789 \mu m^2$, 1.45 mW) technology nodes at 100 MHz with zero timing slack. All physical design signoff checks (DRC, Connectivity, Antenna) passed cleanly. A dedicated web application (BFV Encryption Gateway) was developed using Next.js and Supabase to provide a modern client-side interface for managing the encrypted image transfer workflow with role-based access control.

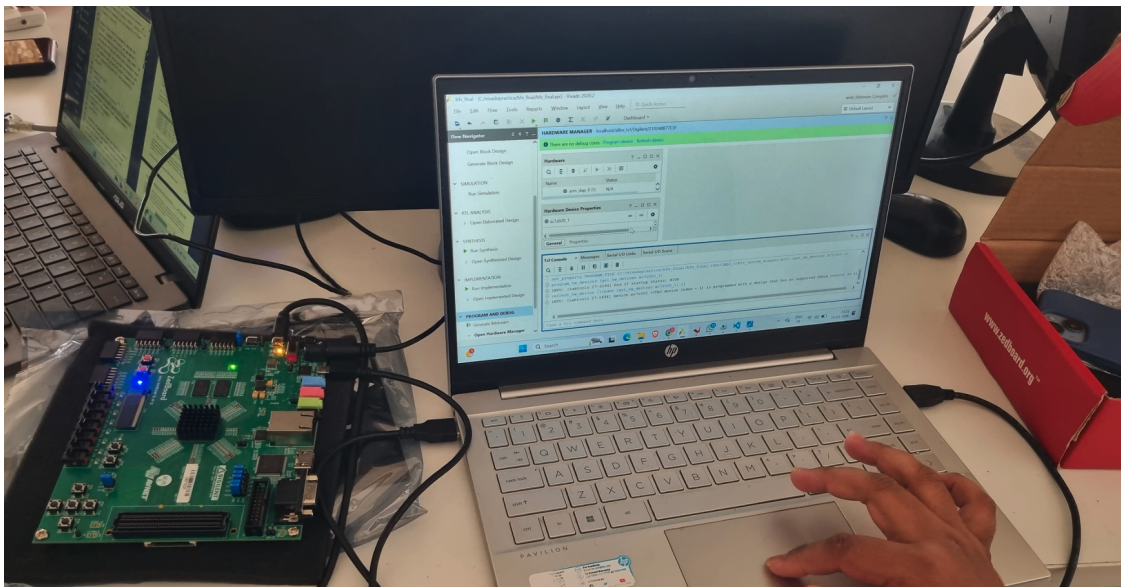


Figure 37: Final aggregated system architecture encapsulating the secure transition from IoT hardware acquisition modules to fully encrypted cloud analytic endpoints.

9.2 Future Scope

1. **ASIC Tape-Out:** Submit the GDSII layout for fabrication at a foundry to manufacture a standalone edge-device SoC chip.
2. **Higher Resolution Images:** Scale the polynomial degree N to support 128×128 (16,384 pixels) and 256×256 (65,536 pixels) images by implementing memory hierarchy with external DRAM.
3. **Color Image Support:** Extend the encryption pipeline to handle RGB images (3 channels $\times$ 8 bits) by parallelizing three BFV encryption cores.
4. **NTT-Based Polynomial Multiplication:** Replace the current coefficient-level multiplication with Number Theoretic Transform (NTT) based polynomial multiplication for supporting batched homomorphic operations on multiple pixels simultaneously.
5. **Ethernet TCP/IP Integration:** Add an Ethernet MAC IP to the SoC for direct network transmission, entirely eliminating the need for an intermediary host PC and UART bridge.
6. **Homomorphic Operations on Cloud:** Implement homomorphic addition and multiplication operations on the cloud side, enabling actual computation on encrypted data (e.g., image filtering, contrast adjustment) without decryption.
7. **5nm/7nm Technology Migration:** Migrate the design to advanced FinFET technology nodes for significantly higher clock frequencies (> 1 GHz) and sub-milliwatt power consumption.
8. **Post-Quantum Security Hardening:** Integrate True Random Number Generators (TRNG) to replace the LFSR-based PRNG and add formal security proofs against quantum adversaries.
9. **Real-Time Video Encryption:** Extend the system to support real-time video streams by implementing frame-level pipelining with double-buffered pixel memories.

10 Appendix A: FPGA Hardware RTL Codes

10.1 Modular Adder (bfv_mod_adder.v)

```

1 //=====
2 // bfv_mod_adder.v
3 // Modular Adder: result = (a + b) mod Q
4 //=====
5 `timescale 1ns/1ps
6
7 module bfv_mod_adder #(
8     parameter W = 14,
9     parameter Q = 12289
10 )
11     input  wire [W-1:0] a,
12     input  wire [W-1:0] b,
13     output wire [W-1:0] result
14 );
15     localparam [W:0] Q_EXT = Q;
16     wire [W:0] sum;
17     wire [W:0] sum_minus_q;
18
19     assign sum          = {1'b0, a} + {1'b0, b};
20     assign sum_minus_q = sum - Q_EXT;
21
22     assign result = (sum >= Q_EXT) ? sum_minus_q[W-1:0] : sum[W-1:0];
23 endmodule

```

10.2 Modular Multiplier (bfv_mod_mult.v)

```

1 //=====
2 // bfv_mod_mult.v
3 // Modular Multiplier: result = (a * b) mod Q
4 // 14-bit x 14-bit -> 28-bit product, then reduce mod Q.
5 // Vivado maps this to DSP48 blocks automatically.
6 //=====
7 `timescale 1ns/1ps
8
9 module bfv_mod_mult #(
10     parameter W = 14,
11     parameter Q = 12289
12 )
13     input  wire [W-1:0] a,
14     input  wire [W-1:0] b,
15     output wire [W-1:0] result
16 );
17     wire [2*W-1:0] product;
18
19     assign product = a * b;
20     assign result  = product % Q;
21 endmodule

```

10.3 Key Generation Core (bfv_keygen.v)

```

1 //=====
2 // bfv_keygen.v
3 // BFV Key Generation
4 // PK1 = [ -(a * SK + e) ] mod Q
5 // PK2 = a
6 //=====
7 `timescale 1ns/1ps
8
9 module bfv_keygen #(

```

```

10     parameter W = 14,
11     parameter Q = 12289
12 )(
13     input  wire [W-1:0] sk,
14     input  wire [W-1:0] a,
15     input  wire [W-1:0] e,
16     output wire [W-1:0] pk1,
17     output wire [W-1:0] pk2
18 );
19     // Step 1: a * SK mod Q
20     wire [W-1:0] a_times_sk;
21     bfv_mod_mult #(.W(W), .Q(Q)) u_mult_ask (
22         .a(a), .b(sk), .result(a_times_sk)
23     );
24
25     // Step 2: (a*SK + e) mod Q
26     wire [W-1:0] ask_plus_e;
27     bfv_mod_adder #(.W(W), .Q(Q)) u_add_e (
28         .a(a_times_sk), .b(e), .result(ask_plus_e)
29     );
30
31     // Step 3: PK1 = -(a*SK + e) mod Q
32     localparam [W-1:0] Q_W = Q;
33     assign pk1 = (ask_plus_e == {W{1'b0}}) ? {W{1'b0}} : (Q_W - ask_plus_e);
34
35     // PK2 = a
36     assign pk2 = a;
37 endmodule

```

10.4 Encryption Core (bfv_encrypt_core.v)

```

1 //=====
2 // bfv_encrypt_core.v
3 // C1 = [ PK1*u + e1 + Delta*M ] mod Q
4 // C2 = [ PK2*u + e2          ] mod Q
5 //=====
6 `timescale 1ns/1ps
7
8 module bfv_encrypt_core #(
9     parameter W      = 14,
10    parameter Q       = 12289,
11    parameter MSG_W   = 8,
12    parameter T       = 256
13 )(
14     input  wire [W-1:0] pk1,
15     input  wire [W-1:0] pk2,
16     input  wire [W-1:0] u,
17     input  wire [W-1:0] e1,
18     input  wire [W-1:0] e2,
19     input  wire [MSG_W-1:0] msg,
20     output wire [W-1:0] c1,
21     output wire [W-1:0] c2
22 );
23     localparam [W-1:0] DELTA = Q / T;
24     wire [W-1:0] msg_w = {(W-MSG_W){1'b0}}, msg};
25
26     // C1 computation
27     wire [W-1:0] pk1_u;
28     bfv_mod_mult #(.W(W), .Q(Q)) u_mult_pk1u (
29         .a(pk1), .b(u), .result(pk1_u) );
30
31     wire [W-1:0] delta_m;
32     bfv_mod_mult #(.W(W), .Q(Q)) u_mult_dm (
33         .a(DELTA), .b(msg_w), .result(delta_m) );
34
35     wire [W-1:0] pk1u_e1;

```

```

36     bfv_mod_adder #(.W(W), .Q(Q)) u_add_e1 (
37         .a(pk1_u), .b(e1), .result(pk1u_e1) );
38
39     bfv_mod_adder #(.W(W), .Q(Q)) u_add_dm (
40         .a(pk1u_e1), .b(delta_m), .result(c1) );
41
42     // C2 computation
43     wire [W-1:0] pk2_u;
44     bfv_mod_mult #(.W(W), .Q(Q)) u_mult_pk2u (
45         .a(pk2), .b(u), .result(pk2_u) );
46
47     bfv_mod_adder #(.W(W), .Q(Q)) u_add_e2 (
48         .a(pk2_u), .b(e2), .result(c2) );
49 endmodule

```

10.5 Top-Level FSM Controller (bfv_top.v – FPGA Version)

```

1 //=====
2 // bfv_top.v -- BFV Top Level with FSM (Area-Time Tradeoff)
3 // ONE shared encrypt core reused for all 4096 pixels sequentially.
4 // Total clocks = 16,386 @100MHz = ~164 us
5 //=====
6 `timescale 1ns/1ps
7
8 module bfv_top #(
9     parameter W      = 14,   parameter Q      = 12289,
10    parameter MSG_W   = 8,    parameter T      = 256,
11    parameter N_PIX   = 4096, parameter ADDR_W = 12
12 )(
13     input  wire      clk, rst_n, start,
14     input  wire [W-1:0] sk, a_key, e_key,
15     output reg        done,
16     output reg [2:0]   state_out,
17     output reg [W-1:0] dbg_pk1, dbg_pk2,
18     output reg [ADDR_W-1:0] dbg_pix_cnt
19 );
20
21 (* ram_style = "block" *)
22 reg [MSG_W-1:0] pixel_mem [0:N_PIX-1];
23 (* ram_style = "block" *)
24 reg [W-1:0] enc_c1 [0:N_PIX-1];
25 (* ram_style = "block" *)
26 reg [W-1:0] enc_c2 [0:N_PIX-1];
27 (* ram_style = "block" *)
28 reg [2*W-1:0] enc_combined [0:N_PIX-1];
29
30 localparam [2:0]
31     S_IDLE=0, S_KEYGEN=1, S_LOAD_PIX=2,
32     S_COMPUTE=3, S_STORE_CT=4, S_NEXT_PIX=5, S_DONE=6;
33
34 reg [2:0] state;
35 reg [ADDR_W-1:0] pix_cnt;
36 localparam [ADDR_W-1:0] LAST_PIX = N_PIX - 1;
37
38 reg [W-1:0] pk1_r, pk2_r;
39 wire [W-1:0] pk1_comb, pk2_comb;
40
41 bfv_keygen #(.W(W), .Q(Q)) u_keygen (
42     .sk(sk), .a(a_key), .e(e_key),
43     .pk1(pk1_comb), .pk2(pk2_comb));
44
45 // 16-bit LFSR
46 reg [15:0] lfsr;
47 wire lfsr_fb = lfsr[15]^lfsr[13]^lfsr[12]^lfsr[10];
48 always @(posedge clk)
49     if (!rst_n) lfsr <= 16'hACE1;
50     else       lfsr <= {lfsr[14:0], lfsr_fb};

```

```

50
51 wire [1:0] u_sel = lfsr[1:0];
52 wire [W-1:0] u_val = (u_sel==2'b01) ? 14'd1 :
53                     (u_sel==2'b10) ? 14'd12288 : 14'd0;
54 wire [W-1:0] e1_val = (lfsr[4:0]>10) ? 14'd10
55                     : {{W-5{1'b0}},lfsr[4:0]};
56 wire [W-1:0] e2_val = (lfsr[9:5]>10) ? 14'd10
57                     : {{W-5{1'b0}},lfsr[9:5]};
58
59 reg [MSG_W-1:0] cur_msg;
60 wire [W-1:0] c1_w, c2_w;
61
62 bfv_encrypt_core #(.W(W),.Q(Q),.MSG_W(MSG_W),.T(T))
63 u_enc (.pk1(pk1_r),.pk2(pk2_r),
64       .u(u_val),.e1(e1_val),.e2(e2_val),
65       .msg(cur_msg),.c1(c1_w),.c2(c2_w));
66
67 always @(posedge clk) begin
68     if (!rst_n) begin
69         state<=S_IDLE; pix_cnt<=0; done<=0;
70         pk1_r<=0; pk2_r<=0; cur_msg<=0;
71     end else begin
72         state_out <= state;
73         case (state)
74             S_IDLE:    if(start) begin done<=0;
75                       state<=S_KEYGEN; end
76             S_KEYGEN:  begin pk1_r<=pk1_comb;
77                       pk2_r<=pk2_comb; pix_cnt<=0;
78                       state<=S_LOAD_PIX; end
79             S_LOAD_PIX: begin cur_msg<=pixel_mem[pix_cnt];
80                       state<=S_COMPUTE; end
81             S_COMPUTE: state<=S_STORE_CT;
82             S_STORE_CT: begin
83                       enc_c1[pix_cnt]<=c1_w;
84                       enc_c2[pix_cnt]<=c2_w;
85                       enc_combined[pix_cnt]<={c1_w,c2_w};
86                       state<=S_NEXT_PIX; end
87             S_NEXT_PIX: if(pix_cnt==LAST_PIX) state<=S_DONE;
88                       else begin pix_cnt<=pix_cnt+1;
89                       state<=S_LOAD_PIX; end
90             S_DONE:    begin done<=1;
91                       if(start) begin done<=0;
92                       state<=S_KEYGEN; end end
93             default:   state<=S_IDLE;
94         endcase
95     end
96 end
97 endmodule

```

11 Appendix B: Software and Interfacing Scripts

11.1 Image Preprocessing Script (Image_converter.py)

```

1 import numpy as np
2 from PIL import Image
3
4 def process_image_for_fpga(image_path, output_file):
5     img = Image.open(image_path)
6     img_gray = img.convert('L')
7     img_resized = img_gray.resize((64, 64))
8     pixel_array = np.array(img_resized).flatten()
9     with open(output_file, 'w') as f:
10         for pixel in pixel_array:
11             f.write(f"{pixel:02x}\n")
12     print(f"Processed {len(pixel_array)} pixels. Saved to {output_file}")
13     return pixel_array
14
15 process_image_for_fpga("Pikachu.png", "image_pixels.mem")

```

11.2 Vitis C Memory Header Converter (convert_image.py)

```

1 import os
2 input_file = "image_pixels.mem"
3 output_file = "image_data.h"
4 print(f"Reading {input_file}...")
5 try:
6     with open(input_file, "r") as f:
7         hex_lines = [line.strip() for line in f.readlines() if line.strip()]
8     if len(hex_lines) != 4096:
9         print(f"WARNING: Expected 4096 pixels, found {len(hex_lines)}.")
10    with open(output_file, "w") as f:
11        f.write("/* Auto-generated 64x64 image data for Vitis */\n")
12        f.write("#ifndef IMAGE_DATA_H\n#define IMAGE_DATA_H\n\n")
13        f.write(f"const unsigned char real_image[{len(hex_lines)}] = {{\n    ")
14        for i, hex_val in enumerate(hex_lines):
15            f.write(f"0x{hex_val}, ")
16            if (i + 1) % 16 == 0:
17                f.write("\n    ")
18        f.write("\n};\n\n#endif\n")
19    print(f"SUCCESS! Generated {output_file} with {len(hex_lines)} pixels.")
20 except FileNotFoundError:
21    print(f"ERROR: Could not find {input_file}.")

```

11.3 UART Capture Script (capture_uart.py)

```

1 import serial, os, sys
2
3 COM_PORT = "COM6"
4 BAUD_RATE = 115200
5 N_PIXELS = 4096
6 SAVE_DIR = "./received"
7 os.makedirs(SAVE_DIR, exist_ok=True)
8
9 ser = serial.Serial(COM_PORT, BAUD_RATE, timeout=60)
10 combined_data, capturing, count = [], False, 0
11
12 while True:
13     raw = ser.readline()
14     line = raw.decode('utf-8', errors='ignore').strip()
15     if not line: continue
16     if "%BFV_START%" in line:
17         capturing, combined_data, count = True, [], 0

```

```

18         continue
19     if "%BFV_END%" in line:
20         capturing = False; break
21     if capturing and len(line) == 7:
22         try:
23             combined_data.append(int(line,16)); count += 1
24         except ValueError: pass
25 ser.close()
26
27 if len(combined_data) == N_PIXELS:
28     outpath = os.path.join(SAVE_DIR, "output_combined.mem")
29     with open(outpath, 'w') as f:
30         for val in combined_data:
31             f.write(f"{val:07x}\n")
32     print(f"File saved: {outpath}")

```

11.4 GitHub Cloud Sync Script (push_to_gist.py)

```

1 import requests, json, os, sys
2
3 GITHUB_TOKEN = "YOUR_SECURE_TOKEN_HERE"
4 FILE_PATH = "./received/output_combined.mem"
5
6 if not os.path.exists(FILE_PATH):
7     print(f"ERROR: Cannot find {FILE_PATH}"); sys.exit(1)
8 with open(FILE_PATH, 'r') as f:
9     file_content = f.read()
10
11 payload = {
12     "description": "BFV Encrypted 64x64 Image (FPGA Hardware)",
13     "public": True,
14     "files": {"output_combined.mem": {"content": file_content}}
15 }
16 headers = {
17     "Authorization": f"token {GITHUB_TOKEN}",
18     "Accept": "application/vnd.github.v3+json"
19 }
20 response = requests.post(
21     "https://api.github.com/gists",
22     headers=headers, data=json.dumps(payload))
23
24 if response.status_code == 201:
25     gist_url = response.json().get("html_url")
26     print(f"SUCCESS! Hosted at: {gist_url}")
27 else:
28     print(f"FAILED. HTTP Error {response.status_code}")

```

11.5 Decryption and Verification Script (decrypt.py)

```

1 import os, sys
2 import numpy as np
3 from PIL import Image
4
5 Q, T, SK = 12289, 256, 1
6 MEM_FILE = sys.argv[1] if len(sys.argv) > 1 else "./received/output_combined.mem"
7 IMG_FILE = sys.argv[2] if len(sys.argv) > 2 else "./image_pixels.mem"
8
9 def bfv_decrypt(c1, c2):
10     inner = (c1 + c2 * SK) % Q
11     return ((inner * T) + (Q // 2)) // Q % T
12
13 # Load encrypted data
14 c1_list, c2_list = [], []
15 with open(MEM_FILE) as f:

```



```
16     for line in f:
17         line = line.strip()
18         if not line: continue
19         val = int(line, 16)
20         c1_list.append((val >> 14) & 0x3FFF)
21         c2_list.append( val          & 0x3FFF)
22
23 n = len(c1_list)
24 decrypted = [bfv_decrypt(c1_list[i], c2_list[i]) for i in range(n)]
25
26 # Load original for comparison
27 with open(IMG_FILE) as f:
28     expected = [int(l.strip(), 16) for l in f if l.strip()]
29
30 correct = sum(1 for a,b in zip(decrypted, expected) if a==b)
31 print(f"RESULT: {correct}/{n} pixels correct")
32 if correct == n:
33     print("PERFECT MATCH! Image verified!")
34
35 # Save reconstructed image
36 if n >= 4096:
37     arr = np.array(decrypted[:4096], dtype=np.uint8).reshape(64,64)
38     img = Image.fromarray(arr, 'L').resize((512,512), Image.NEAREST)
39     img.save("./received/decrypted_image.png")
40     print("Saved decrypted_image.png")
```

12 Appendix C: ASIC Synthesis Scripts

12.1 Genus Synthesis Script (run_genus_pipelined_180.tcl)

```

1 # Cadence Genus Synthesis Script - Pipelined BFV Core (180nm)
2
3 # Setup 180nm Library Paths
4 set_db init_lib_search_path /home/install/FOUNDRY/digital/180nm/dig/lib/
5 set_db library slow.lib
6
7 # Read Verilog files in bottom-up dependency order
8 read_hdl {
9     bfv_mod_adder.v
10    bfv_mod_mult.v
11    bfv_keygen.v
12    bfv_encrypt_core.v
13    bfv_top.v
14 }
15
16 # Elaborate top-level module
17 elaborate bfv_top
18
19 # Apply optimized constraints
20 read_sdc ./bfv_pipelined_180nm.sdc
21
22 # Power optimization goals
23 set_max_leakage_power 0.0
24 set_max_dynamic_power 0.0
25
26 # Synthesis effort levels
27 set_db syn_generic_effort high
28 set_db syn_map_effort high
29 set_db syn_opt_effort high
30
31 # Run Synthesis
32 syn_generic
33 syn_map
34 syn_opt
35
36 # Export Netlist and SDC for Innovus
37 write_hdl > bfv_180nm_pipelined_netlist.v
38 write_sdc > bfv_180nm_pipelined_output.sdc
39
40 # Generate Reports
41 report timing > bfv_180nm_pipelined_timing.rpt
42 report power > bfv_180nm_pipelined_power.rpt
43 report area > bfv_180nm_pipelined_area.rpt
44 report gates > bfv_180nm_pipelined_gates.rpt

```

12.2 SDC Timing Constraints (bfv_pipelined_180nm.sdc)

```

1 # BFV SoC - Timing Constraints
2 # Target Frequency: 100 MHz (10.0 ns period)
3
4 create_clock -name clk -period 10.0 [get_ports clk]
5
6 set_clock_uncertainty -setup 0.2 [get_clocks clk]
7 set_clock_uncertainty -hold 0.1 [get_clocks clk]
8 set_clock_transition 0.2 [get_clocks clk]
9
10 set_input_delay 0.5 -max -clock clk [remove_from_collection \
11     [all_inputs] [get_ports clk]]
12 set_input_delay 0.1 -min -clock clk [remove_from_collection \
13     [all_inputs] [get_ports clk]]

```

```

14
15 set_output_delay 0.5 -max -clock clk [all_outputs]
16 set_output_delay 0.1 -min -clock clk [all_outputs]
17
18 set_input_transition 0.5 [all_inputs]
19 set_load 2.0 [all_outputs]

```

12.3 Testbench (tb_bfv_top.v)

```

1 //=====
2 // tb_bfv_top.v -- BFV Testbench
3 // Loads image_pixels.mem, encrypts all 4096 pixels,
4 // writes output_c1.mem, output_c2.mem, output_combined.mem
5 //=====
6 `timescale 1ns/1ps
7
8 module tb_bfv_top;
9     localparam W=14, Q=12289, MSG_W=8, T=256, N_PIX=4096, ADDR_W=12;
10
11     reg clk, rst_n, start;
12     reg [W-1:0] sk, a_key, e_key;
13     wire done;
14     wire [2:0] state_out;
15     wire [W-1:0] dbg_pk1, dbg_pk2;
16     wire [ADDR_W-1:0] dbg_pix_cnt;
17
18     bfv_top #(W(W),.Q(Q),.MSG_W(MSG_W),.T(T),
19             .N_PIX(N_PIX),.ADDR_W(ADDR_W)) dut (
20         .clk(clk),.rst_n(rst_n),.start(start),
21         .sk(sk),.a_key(a_key),.e_key(e_key),
22         .done(done),.state_out(state_out),
23         .dbg_pk1(dbg_pk1),.dbg_pk2(dbg_pk2),
24         .dbg_pix_cnt(dbg_pix_cnt));
25
26     initial clk = 0;
27     always #5 clk = ~clk; // 100 MHz
28
29     integer i;
30     initial begin
31         $readmemh("image_pixels.mem", dut.pixel_mem);
32         rst_n=0; start=0; sk=14'd1; a_key=14'd9847; e_key=14'd3;
33         repeat(6) @(posedge clk);
34         @(negedge clk); rst_n=1;
35         repeat(2) @(posedge clk);
36         @(negedge clk); start=1;
37         @(posedge clk);
38         @(negedge clk); start=0;
39
40         @(posedge done);
41         repeat(4) @(posedge clk);
42
43         $writememh("output_c1.mem", dut.enc_c1);
44         $writememh("output_c2.mem", dut.enc_c2);
45         $writememh("output_combined.mem", dut.enc_combined);
46
47         $display("ENCRYPTION COMPLETE at %0t ns", $time);
48         for (i=0; i<16; i=i+1)
49             $display("%04d 0x%02x C1=0x%04x C2=0x%04x",
50                 i, dut.pixel_mem[i], dut.enc_c1[i], dut.enc_c2[i]);
51
52         repeat(5) @(posedge clk);
53         $finish;
54     end
55
56     initial begin #2_000_000; $finish; end
57     initial begin $dumpfile("bfv_sim.vcd"); $dumpvars(0,tb_bfv_top); end

```

References

1. **Inferati Inc.** "Introduction to the BFV FHE Scheme." Washington, USA. Available: <https://faculty.kfupm.edu.sa/coe/mfelemban/SEC595/References/Introduction%20to%20the%20BFV%20FHE%20Scheme.pdf>
2. **Mareta, R., Satriawan, A., and Lee, H.** (2025). "High Throughput Arithmetic Computing Unit for BFV Homomorphic Encryption." *IEEE Open Journal of Circuits and Systems*. DOI: 10.1109/OJCAS.2025.3581188.
3. **Amdouni, R., et al.** (2024). "FPGA Implementation of Robust and Secure Transmission Cryptosystem for Satellite Images." *IEEE Access*. DOI: 10.1109/ACCESS.2024.3444732.
4. **Brakerski, Z.** (2012). "Fully Homomorphic Encryption without Bootstrapping and with n-bit Security." *Electronic Colloquium on Computational Complexity (ECCC)*.
5. **Fan, J., and Vercauteren, F.** (2012). "Somewhat Homomorphic Encryption over $\mathbb{Z}_q$." *Cryptology ePrint Archive*.
6. **Gentry, C.** (2009). "Fully Homomorphic Encryption Using Ideal Lattices." *ACM Symposium on Theory of Computing (STOC)*.
7. **Microsoft SEAL (v4.1).** "Simple Encrypted Arithmetic Library." Available: <https://github.com/microsoft/SEAL>.
8. **Roy, S. S., and Vercauteren, F.** (2020). "NTRU and BFV on the Zynq-7000 SoC." *Journal of Cryptographic Engineering*.
9. **Xilinx Inc.** (2021). "Zynq-7000 SoC Data Sheet: Overview." DS190.
10. **Cadence Design Systems.** (2020). "Genus Synthesis Solution User Guide."
11. **Cadence Design Systems.** (2020). "Innovus Implementation System User Guide."
12. **Cousins, D. B., and Rohloff, K.** (2017). "A Scalable Architecture for Homomorphic Encryption." *IEEE High Performance Extreme Computing Conference (HPEC)*.
13. **Mert, A. C., Öztürk, E., and Savaş, E.** (2020). "Highly Parallel NTT-based Accelerator for BFV Homomorphic Encryption." *IEEE International Symposium on Circuits and Systems (ISCAS)*.
14. **Öztürk, E., et al.** (2015). "A Retargetable Accelerator for Homomorphic Encryption." *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*.
15. **TSMC.** "180nm and 90nm Standard Cell Library Documentation."